

pyAmpliCol 0.1.0

Methodology and Performance Report

Rikkert Frederix

Valentin Hirschi

Timea Vitos

Release 0.1.0

Abstract

This report defines the public theory, validation, and performance methodology for `pyAmpliCol`, release 0.1.0. It defines the canonical process coverage for the built-in Standard Model, the same physics loaded as a UFO/JSON Standard Model (UFO-SM), a scalar contact model, and a scalar-gravity model. Its tables present reproducible measurements tied to an effective configuration, execution environment, generated artifact, and independent-reference provenance.

Contents

1	Scope	2
2	Current Recursion and Model Lowering	2
3	Colour and Runtime Semantics	3
4	Benchmark Methodology	3
5	Standard-Model Process Matrices	5
6	Z-plus-Jets Evaluator Ladders	35
7	Colour-Singlet Model Ladders	44
8	Interpretation	44
9	Worked $d\bar{d} \rightarrow Zgg$ Example	45
10	The $d\bar{d} \rightarrow Zgg$ DAG and Rusticol	48
11	Reusable Leading-Colour Execution Layouts	52
12	Prepared-Model Eager DAG Execution	54
13	Arbitrary UFO and Serialized-JSON Models	58

1 Scope

`pyAmpliCol` generates tree-level scattering-amplitude artifacts and evaluates them through the Rusticol runtime. The Python layer resolves models and processes. Compiled mode lowers a shared-current directed acyclic graph (DAG) into process-specific evaluators. Eager mode lowers the same proven physics graph into invocation tables over a prepared model-kernel pack. Recurrence mode constructs compact current, contribution, and closure schedules without materializing a generic process DAG. All three routes write versioned process artifacts and preserve the same public physics contract for summed and resolved evaluation from Python, C11, C++17, Fortran 2008, and Rust 2021 clients.

The report has three purposes:

1. specify the observables and conditions required for a publishable benchmark;
2. preserve a stable matrix of representative process families and colour approximations; and
3. separate measured data from prose by rendering every result table from validated measurement records.

An `N/A` entry means that the current result set has no recorded measurement for that slot. It is neither a zero, a failure, nor a value carried over from another implementation. Diagnostic status labels, when present, identify a recorded campaign stop rather than a physics value. A cell marked `SKIP` records a deliberately held high-multiplicity measurement whose estimated or observed resource profile falls outside the campaign envelope.

2 Current Recursion and Model Lowering

2.1 Shared-current DAG

For an external subset I , particle state t , and local spin or flow label χ , an off-shell current has the schematic recursion

$$J_{t,\chi}^\alpha(I) = P^\alpha_\beta(t, p_I) \sum_{I=A \dot{\cup} B} V^\beta_{\gamma\delta}(t_A, t_B \rightarrow t) J_{t_A, \chi_A}^\gamma(A) J_{t_B, \chi_B}^\delta(B), \quad p_I = \sum_{i \in I} p_i. \quad (1)$$

Current identity includes the particle, external labels, helicity ancestry, chirality or spin state, flavour and charge flow, colour state, momentum mask, and any auxiliary-current kind. Contributions with the same complete identity share one node. A topological sweep therefore evaluates each retained current once per phase-space point and reuses it in all dependent amplitudes.

External UFO or serialized JSON sources are compiled to the same canonical model representation as the process generator consumes. Particle metadata, interactions, coupling orders, Lorentz tensors, colour tensors, propagators, and runtime parameters come from that representation. UFO directories contain trusted executable Python; serialized JSON is the portable data-only source.

Optimizations on the external-model path are proof-gated rather than selected by particle names or PDG values. Exact normalized colour and Lorentz expressions, propagator metadata, and coupling relations certify current reflection, helicity, and reusable-kernel identities. For overlapping Standard-Model validation processes, the resulting current, interaction, amplitude-root, and colour-sector counts must match the built-in compatibility model after applying the same independently inferred perturbative-order envelope. An external compiler may retain fewer evaluator calls only when an additional exact kernel-equivalence certificate proves that reuse.

2.2 Contact terms and spin two

A vertex with valence above three is lowered to a deterministic tree of non-propagating auxiliary currents. The construction inserts the original coupling once and introduces no pole. Direct contraction of the original contact tensor is the validation oracle for the scalar-contact ladder.

The scalar-gravity ladder exercises rank-two external states and momentum-dependent interactions. For a massless graviton, the helicity-two polarization tensor is represented as

$$\epsilon_{\pm 2}^{\mu\nu} = \epsilon_\pm^\mu \epsilon_\pm^\nu. \quad (2)$$

Validation checks symmetry, transversality, tracelessness, normalization, and agreement between ordinary and higher-precision evaluation.

3 Colour and Runtime Semantics

The process matrices use three colour contracts:

LC	Leading-colour ordered amplitudes with physical colour-flow selectors.
NLC	The leading term plus the first retained $1/N_c^2$ -suppressed contribution.
Full	The complete supported sparse colour metric for the generated amplitude basis.

For NLC, “retained” applies to a colour-matrix *entry*, not to a truncated numerical coefficient. Write the exact overlap of two colour-flow tensors as

$$C_{ab}(N_c) = \sum_p c_p^{(ab)} N_c^p, \quad p_{ab} = \max\{p : c_p^{(ab)} \neq 0\}, \quad (3)$$

and let P be the leading power of the complete colour basis. The sparse NLC metric used by `build_color_contraction_plan()` in `src/pyamplicol/color/contraction.py` is

$$C_{ab}^{\text{NLC}} = \begin{cases} C_{ab}(3), & p_{ab} \geq P - 2, \\ 0, & p_{ab} < P - 2. \end{cases} \quad (4)$$

Thus every qualifying entry keeps all of its finite- N_c terms. For three open fundamental lines and one adjoint, $P = 4$. Representative retained off-diagonal overlaps are

exact overlap	highest power	NLC at $N_c = 3$	full at $N_c = 3$
$-N_c^3 + N_c$	3	-24	-24
$N_c^2 - 1$	2	8	8

The implementation in `src/pyamplicol/color/contraction_factors.py` carries these powers as exact rational coefficients. The upper-triangular runtime plan applies a separate factor of two to off-diagonal interference terms; that storage factor must not be confused with the colour coefficient in Eq. (4).

The optimized `Runtime.evaluate()` path returns one summed matrix element per point.

The resolved method `Runtime.evaluate_resolved()` exposes LC values with axes (`point`, `helicity`, `flow`). NLC and full-colour artifacts expose one already contracted colour component per helicity. Summing all non-point axes must reproduce the optimized total. Public selectors use physical helicity and flow identifiers; graph-local colour-sector identifiers are not report inputs.

All language bindings load the same process manifest and evaluator payloads. Cross-language agreement therefore validates the runtime interfaces and data layout. A separately generated reference is required for an independent physics comparison.

The default JIT artifact embeds a self-contained SymJIT application for f64 evaluation. Rusticol executes that payload through the separate MIT-licensed SymJIT runtime without importing Symbolica or consulting its runtime license state. Compiled ASM and C++ f64 payloads are likewise loaded directly by the native core. Python evaluation above f64 precision uses the retained Symbolica evaluator state and therefore requires Symbolica at runtime.

4 Benchmark Methodology

4.1 Configuration contract

Each measurement begins from an immutable resolved configuration. Unless a table variant says otherwise, the report contract uses evaluator batch size 128, output chunk size 512, stage-local parameter layout, ten Horner iterations,

backend-default common-pair elimination, a twenty-second benchmark target, two warmup runs, and at least five timed samples. Compiled process matrices request JIT O3. Eager-DAG and recurrence matrices use portable JIT O2 prepared models. Explicitly labelled comparison rows in the dedicated Z ladder may select JIT O1, JIT O3, ASM O3, or C++ O3 instead. Requested and effective configurations are retained separately so backend or resource adjustments remain visible.

Generation time is wall time for one explicit `Generator.generate()` operation starting from the resolved model input required by the selected execution mode and ending with a committed, loadable process artifact. It includes process expansion, mode-specific process lowering, validation-point preparation, artifact publication, and API-bundle emission. Compiled mode additionally includes process-specific evaluator construction and backend compilation. Eager mode includes DAG extraction, invocation-plan lowering, and runtime-container serialization. Recurrence mode includes compact schedule construction and serialization. Resolving or preparing a reusable built-in or UFO/JSON model bundle occurs before the process-generation timer and is recorded separately as model-level provenance. Existing artifacts may be profiled, but their generation slot remains N/A unless a matching generation measurement is available.

Runtime measurements use `BenchmarkRunner` and the summed `Runtime.evaluate()` observable. The reported wall time begins inside Rusticol after caller-language array conversion and covers repeated ordinary core evaluations, including source preparation, numerical execution, output assignment, selector work, and reduction. The secondary native execution submetric is mode-specific: generated stage and amplitude evaluation for compiled mode, invocation-table execution for eager mode, and compact recurrence-schedule execution for recurrence mode. Python/NumPy packing and language-binding conversion are excluded from both reported quantities. A measurement records sample count, time per point, uncertainty, effective settings, software revision, artifact identity, hardware, and execution environment. Comparisons are valid only for the same process, colour accuracy, parameters, phase-space ensemble, precision, selectors, and normalization.

4.2 Benchmark platform

The populated measurements in this report were produced on the cluster host `itphlies.clan` from the Nix developer shell, not from a scheduler-visible batch partition in the reporting environment. The node reports Linux 6.12.63 on NixOS 26.05, x86_64, glibc 2.42. The processor is a dual-socket AMD EPYC 9754 system (Genoa/Zen 4): 256 physical cores are visible across eight NUMA nodes, with 384 logical CPUs online in this session. The advertised instruction-set families include SSE through SSE4.2, FMA, AVX, AVX2, AVX-512 foundation/vector, VN-NI/BF16, AES/SHA, BMI1/2, ADX, VAES, and carry-less multiplication. Installed memory reported by the kernel is 1188419660 kB ($\simeq 1.11$ TiB), with no configured swap.

All report measurements use the repository’s developer installation through `.venv/bin/python` inside `nix develop`. The recorded toolchain is Python 3.11.15 with NumPy 2.4.2, Rust 1.89.0, Cargo 1.89.0, GCC/G++/GNU Fortran 15.2.0, and pdfTeX 3.141592653-2.6-1.40.27 from TeX Live 2025. Unless a cell’s manifest states otherwise, generation and evaluation use one core and runtime sampling targets twenty seconds. Resource ceilings and timeouts are retained in each measurement record; a limit reached is rendered as a status rather than a timing value. Collection concurrency and scheduler queue time are not part of a cell’s workload. Wall times are native Rusticol repeated-core timings after caller-language momentum packing, and model compilation is recorded separately from process-generation time.

4.3 Numerical validation

The matrix element and its independent reference are evaluated at the same deterministic points with the same parameter card. Shared-core language checks use relative tolerance 10^{-12} and absolute tolerance 10^{-15} . Independent reference comparisons use relative tolerance 10^{-8} , with higher precision used to diagnose unstable points. A populated result must also satisfy the resolved-sum identity and preserve expected structural zeros.

No timing is accepted as a numerical validation result. Conversely, a valid matrix element does not populate generation or runtime fields without a matching timing run. A result enters the report only after its record is internally consistent and its pointwise comparison has the status required by the table.

5 Standard-Model Process Matrices

The process matrices use recurrence as the primary `pyAmpliCol` execution mode. Built-in-SM and UFO-SM recurrence JIT O2 are compared directly with the independent original `AmpliCol` reference. Separate built-in-SM matrices then compare compiled JIT O3 and eager-DAG JIT O2 with the matching built-in-SM recurrence JIT O2 measurement. Baseline measurements are joined when the tables are rendered; they are not copied into candidate result records.

The canonical matrix spans electroweak vector production, charged-current and leptonic channels, heavy-quark production, pure gluons, multi-boson final states, Higgs-associated final states, and three- and four-quark-line cases. Here n is the number of final-state particles. LC coverage retains the declared grid through $n = 9$, with the bounded families ending at the multiplicity stated by the process catalog. NLC and full-colour grids retain $n = 1, \dots, 5$. Cells below a family’s minimum physical multiplicity, above its declared limit, or without a valid measurement are shown explicitly as `N/A`.

The UFO-SM recurrence matrices use the same process definitions, colour contracts, numerical parameters, selectors, and validation points as the built-in-SM recurrence matrices. Their source is the packaged Standard Model loaded through the generic UFO/JSON compiler. This comparison tests model-generic lowering without changing the requested physics or the original `AmpliCol` reference. Compiled and eager process matrices are built-in-only; they measure execution-strategy differences against recurrence without introducing a second model-loading comparison.

Workloads and denominators

For a reported timing quantity T , each multiplier is

$$r_{\text{candidate/baseline}} = \frac{T_{\text{candidate}}}{T_{\text{baseline}}}. \quad (5)$$

Green, orange, and red denote respectively $r < 1$, $1 \leq r < 2$, and $r \geq 2$. Green therefore means that the candidate is faster than the baseline named in the table heading. The recurrence matrices use original `AmpliCol` as the baseline. The compiled and eager matrices use built-in-SM recurrence JIT O2 as the baseline.

Every LC comparison contains two independently generated, complete-coverage artifacts:

- **topology-replay**, profiled with one physical colour flow selected at runtime and all retained helicities summed; and
- **all-flow-union**, profiled with one physical helicity selected at runtime and all retained colour flows summed.

Neither the displayed flow nor the displayed helicity is fixed during generation. Each LC cell reports the two absolute baseline generation times and wall times, followed by layout-matched candidate generation ratios and (`wall|execution`) runtime ratios. Thus compiled and eager generation are compared separately with recurrence topology replay and recurrence all-flow union, rather than against one shared recurrence generation value.

NLC and full colour use one helicity-summed, colour-contracted artifact per mode. Their cells report the absolute baseline generation and wall time, followed by the candidate generation ratio and (`wall|execution`) runtime ratios. No per-flow scalar is defined for these contracted colour accuracies.

The wall quantity is the native Rusticol repeated-evaluation time after caller-language input conversion. The execution quantity is the native mode-specific numerical submetric: compiled stage and amplitude evaluation, eager invocation-table execution, or direct recurrence-schedule execution. Both quantities are divided by the same matching baseline wall time because original `AmpliCol` and cross-mode baselines do not expose an equivalent internal submetric.

Preparing a reusable model-level kernel or recurrence-template bundle is excluded from every process-generation entry and retained as separate provenance. Process-generation timings begin from the prepared input required by the selected mode. Compiled timings still include process-specific evaluator construction and backend compilation.

Pointwise publication requires recurrence to agree with original **AmpliCol** at the independent-reference tolerance. Compiled and eager candidates must agree with recurrence at the stricter shared-core tolerance. Each candidate must also reproduce its own resolved sum; LC comparisons additionally use the same physical flow and helicity contracts in both layouts. A failed validation publishes a diagnostic status but no timing ratio.

Cell and summary layout

contract	baseline entry	candidate entries
LC, upper	topology-replay/all-flow-union generation times in seconds	layout-matched generation ratios
LC, lower	selected-flow/all-flow wall times in microseconds per point	for each workload, (wall execution) ratios
NLC/full, upper	contracted generation time in seconds	candidate generation ratio
NLC/full, lower	contracted wall time in microseconds per point	(wall execution) ratios

Summary rows report the baseline mean over valid candidate–baseline pairs and the ratio of sums $\sum_i T_{\text{candidate},i} / \sum_i T_{\text{baseline},i}$. LC summaries remain separate for topology replay and all-flow union. Missing, unsupported, resource-limited, and validation-failed cells do not contribute to a ratio-of-sums summary.

The following tables appear in catalog order: recurrence before compiled and eager; built-in SM before UFO-SM where both are defined; and LC, NLC, then full colour within each mode.

5.1 Built-in SM recurrence versus AmpliCol LC

Built-in SM recurrence versus AmpliCol LC (block 1 of 3)

ID base process	n=1	n=2	n=3
1 $d\bar{d} \rightarrow Z + (n-1)g$	15.4 / 0.00034 (x1.24) (x9.37e+04) 0.169 / 0.249 (x6.21 x4.88) (x8.89 x3)	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
2 $u\bar{d} \rightarrow W^+ + (n-1)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
3 $d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
4 $u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
5 $d\bar{d} \rightarrow ZZ + (n-2)g$	N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
6 $gg \rightarrow t\bar{t} + (n-2)g$	N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
7 $d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
8 $gg \rightarrow gg + (n-2)g$	N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
9 $d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A	N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
10 $d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A	N/A	N/A
11 $d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A	N/A	N/A
12 $d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A	N/A	N/A
13 $d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A	N/A	N/A
14 $d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A	N/A
summary: generation	15.4 (x1.24) / 0.00034 (x9.37e+04)	N/A / N/A	N/A / N/A
summary: wall	0.169 (x6.21) / 0.249 (x8.89)	N/A / N/A	N/A / N/A

Baseline: original AmpliCol; candidate: recurrence JIT O2. Each cell shows the topology-replay/all-flow-union baseline generation times and wall times, followed by layout-matched candidate/baseline generation and (wall|native execution) ratios.

Built-in SM recurrence versus AmpliCol LC (block 2 of 3)

ID base process	n=4	n=5	n=6
1 $d\bar{d} \rightarrow Z + (n-1)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
2 $u\bar{d} \rightarrow W^+ + (n-1)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
3 $d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
4 $u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
5 $d\bar{d} \rightarrow ZZ + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
6 $gg \rightarrow t\bar{t} + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
7 $d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
8 $gg \rightarrow gg + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
9 $d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
10 $d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
11 $d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
12 $d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
13 $d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
14 $d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
summary: generation	N/A / N/A	N/A / N/A	N/A / N/A
summary: wall	N/A / N/A	N/A / N/A	N/A / N/A

Baseline: original AmpliCol; candidate: recurrence JIT O2. Each cell shows the topology-replay/all-flow-union baseline generation times and wall times, followed by layout-matched candidate/baseline generation and (wall|native execution) ratios.

Built-in SM recurrence versus AmpliCol LC (block 3 of 3)

ID base process	n=7	n=8	n=9
1 $d\bar{d} \rightarrow Z + (n-1)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
2 $u\bar{d} \rightarrow W^+ + (n-1)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
3 $d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
4 $u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
5 $d\bar{d} \rightarrow ZZ + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
6 $gg \rightarrow t\bar{t} + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A
7 $d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
8 $gg \rightarrow gg + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A
9 $d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
10 $d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
11 $d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
12 $d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
13 $d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A
14 $d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A
summary: generation	N/A / N/A	N/A / N/A	N/A / N/A
summary: wall	N/A / N/A	N/A / N/A	N/A / N/A

Baseline: original AmpliCol; candidate: recurrence JIT O2. Each cell shows the topology-replay/all-flow-union baseline generation times and wall times, followed by layout-matched candidate/baseline generation and (wall|native execution) ratios.

5.2 Built-in SM recurrence versus AmpliCol NLC

Built-in SM recurrence versus AmpliCol NLC (block 1 of 2)

ID base process	n=1	n=2	n=3
1 $d\bar{d} \rightarrow Z + (n-1)g$	16 (x1.38) 0.804 (x1.1 x0.97)	N/A N/A N/A N/A	N/A N/A N/A N/A
2 $u\bar{d} \rightarrow W^+ + (n-1)g$	N/A N/A N/A N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
3 $d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
4 $u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
5 $d\bar{d} \rightarrow ZZ + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
6 $gg \rightarrow t\bar{t} + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
7 $d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
8 $gg \rightarrow gg + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
9 $d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A	N/A	N/A N/A N/A N/A
10 $d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A	N/A	N/A
11 $d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A	N/A	N/A
12 $d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A	N/A	N/A
13 $d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A	N/A	N/A
14 $d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A	N/A
summary: generation	16 (x1.38)	N/A	N/A
summary: wall	0.804 (x1.1)	N/A	N/A

Baseline: original AmpliCol; candidate: recurrence JIT O2. Each cell shows the baseline generation and wall time, followed by candidate/baseline generation and (wall|native execution) ratios.

Built-in SM recurrence versus AmpliCol NLC (block 2 of 2)

ID base process	n=4	n=5
1 $d\bar{d} \rightarrow Z + (n-1)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
2 $u\bar{d} \rightarrow W^+ + (n-1)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
3 $d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
4 $u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
5 $d\bar{d} \rightarrow ZZ + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
6 $gg \rightarrow t\bar{t} + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
7 $d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
8 $gg \rightarrow gg + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
9 $d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
10 $d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
11 $d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
12 $d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
13 $d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
14 $d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A
summary: generation	N/A	N/A
summary: wall	N/A	N/A

Baseline: original AmpliCol; candidate: recurrence JIT O2. Each cell shows the baseline generation and wall time, followed by candidate/baseline generation and (wall|native execution) ratios.

5.3 Built-in SM recurrence versus AmpliCol full-colour

Built-in SM recurrence versus AmpliCol full-colour (block 1 of 2)

ID base process	n=1	n=2	n=3
1 $d\bar{d} \rightarrow Z + (n-1)g$	N/A N/A N/A N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
2 $u\bar{d} \rightarrow W^+ + (n-1)g$	N/A N/A N/A N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
3 $d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
4 $u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
5 $d\bar{d} \rightarrow ZZ + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
6 $gg \rightarrow t\bar{t} + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
7 $d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
8 $gg \rightarrow gg + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
9 $d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A	N/A	N/A N/A N/A N/A
10 $d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A	N/A	N/A
11 $d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A	N/A	N/A
12 $d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A	N/A	N/A
13 $d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A	N/A	N/A
14 $d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A	N/A
summary: generation	N/A	N/A	N/A
summary: wall	N/A	N/A	N/A

Baseline: original AmpliCol; candidate: recurrence JIT O2. Each cell shows the baseline generation and wall time, followed by candidate/baseline generation and (wall|native execution) ratios.

Built-in SM recurrence versus AmpliCol full-colour (block 2 of 2)

ID base process	n=4	n=5
1 $d\bar{d} \rightarrow Z + (n-1)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
2 $u\bar{d} \rightarrow W^+ + (n-1)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
3 $d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
4 $u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
5 $d\bar{d} \rightarrow ZZ + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
6 $gg \rightarrow t\bar{t} + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
7 $d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
8 $gg \rightarrow gg + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
9 $d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
10 $d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
11 $d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
12 $d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
13 $d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
14 $d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A
summary: generation	N/A	N/A
summary: wall	N/A	N/A

Baseline: original AmpliCol; candidate: recurrence JIT O2. Each cell shows the baseline generation and wall time, followed by candidate/baseline generation and (wall|native execution) ratios.

5.4 UFO-SM recurrence versus AmpliCol LC

UFO-SM recurrence versus AmpliCol LC (block 1 of 3)

ID base process	n=1	n=2	n=3
1 $d\bar{d} \rightarrow Z + (n-1)g$	15.4 / 0.00034 N/A N/A 0.169 / 0.249 N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
2 $u\bar{d} \rightarrow W^+ + (n-1)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
3 $d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
4 $u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
5 $d\bar{d} \rightarrow ZZ + (n-2)g$	N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
6 $gg \rightarrow t\bar{t} + (n-2)g$	N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
7 $d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
8 $gg \rightarrow gg + (n-2)g$	N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
9 $d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A	N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
10 $d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A	N/A	N/A
11 $d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A	N/A	N/A
12 $d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A	N/A	N/A
13 $d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A	N/A	N/A
14 $d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A	N/A
summary: generation	N/A / N/A	N/A / N/A	N/A / N/A
summary: wall	N/A / N/A	N/A / N/A	N/A / N/A

Baseline: original AmpliCol; candidate: recurrence JIT O2. Each cell shows the topology-replay/all-flow-union baseline generation times and wall times, followed by layout-matched candidate/baseline generation and (wall|native execution) ratios.

UFO-SM recurrence versus AmpliCol LC (block 2 of 3)

ID base process	n=4	n=5	n=6
1 $d\bar{d} \rightarrow Z + (n-1)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
2 $u\bar{d} \rightarrow W^+ + (n-1)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
3 $d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
4 $u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
5 $d\bar{d} \rightarrow ZZ + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
6 $gg \rightarrow t\bar{t} + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
7 $d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
8 $gg \rightarrow gg + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
9 $d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
10 $d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
11 $d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
12 $d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
13 $d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
14 $d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
summary: generation	N/A / N/A	N/A / N/A	N/A / N/A
summary: wall	N/A / N/A	N/A / N/A	N/A / N/A

Baseline: original AmpliCol; candidate: recurrence JIT O2. Each cell shows the topology-replay/all-flow-union baseline generation times and wall times, followed by layout-matched candidate/baseline generation and (wall|native execution) ratios.

UFO-SM recurrence versus AmpliCol LC (block 3 of 3)

ID base process	n=7	n=8	n=9
1 $d\bar{d} \rightarrow Z + (n-1)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
2 $u\bar{d} \rightarrow W^+ + (n-1)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
3 $d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
4 $u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
5 $d\bar{d} \rightarrow ZZ + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
6 $gg \rightarrow t\bar{t} + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A
7 $d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
8 $gg \rightarrow gg + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A
9 $d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
10 $d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
11 $d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
12 $d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
13 $d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A
14 $d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A
summary: generation	N/A / N/A	N/A / N/A	N/A / N/A
summary: wall	N/A / N/A	N/A / N/A	N/A / N/A

Baseline: original AmpliCol; candidate: recurrence JIT O2. Each cell shows the topology-replay/all-flow-union baseline generation times and wall times, followed by layout-matched candidate/baseline generation and (wall|native execution) ratios.

5.5 UFO-SM recurrence versus AmpliCol NLC

UFO-SM recurrence versus AmpliCol NLC (block 1 of 2)

ID base process	n=1	n=2	n=3
1 $d\bar{d} \rightarrow Z + (n-1)g$	16 N/A 0.804 N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
2 $u\bar{d} \rightarrow W^+ + (n-1)g$	N/A N/A N/A N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
3 $d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
4 $u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
5 $d\bar{d} \rightarrow ZZ + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
6 $gg \rightarrow t\bar{t} + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
7 $d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
8 $gg \rightarrow gg + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
9 $d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A	N/A	N/A N/A N/A N/A
10 $d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A	N/A	N/A
11 $d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A	N/A	N/A
12 $d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A	N/A	N/A
13 $d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A	N/A	N/A
14 $d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A	N/A
summary: generation	N/A	N/A	N/A
summary: wall	N/A	N/A	N/A

Baseline: original AmpliCol; candidate: recurrence JIT O2. Each cell shows the baseline generation and wall time, followed by candidate/baseline generation and (wall|native execution) ratios.

UFO-SM recurrence versus AmpliCol NLC (block 2 of 2)

ID base process	n=4	n=5
1 $d\bar{d} \rightarrow Z + (n-1)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
2 $u\bar{d} \rightarrow W^+ + (n-1)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
3 $d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
4 $u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
5 $d\bar{d} \rightarrow ZZ + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
6 $gg \rightarrow t\bar{t} + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
7 $d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
8 $gg \rightarrow gg + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
9 $d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
10 $d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
11 $d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
12 $d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
13 $d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
14 $d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A
summary: generation	N/A	N/A
summary: wall	N/A	N/A

Baseline: original AmpliCol; candidate: recurrence JIT O2. Each cell shows the baseline generation and wall time, followed by candidate/baseline generation and (wall|native execution) ratios.

5.6 UFO-SM recurrence versus AmpliCol full-colour

UFO-SM recurrence versus AmpliCol full-colour (block 1 of 2)

ID base process	n=1	n=2	n=3
1 $d\bar{d} \rightarrow Z + (n-1)g$	N/A N/A N/A N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
2 $u\bar{d} \rightarrow W^+ + (n-1)g$	N/A N/A N/A N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
3 $d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
4 $u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
5 $d\bar{d} \rightarrow ZZ + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
6 $gg \rightarrow t\bar{t} + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
7 $d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
8 $gg \rightarrow gg + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
9 $d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A	N/A	N/A N/A N/A N/A
10 $d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A	N/A	N/A
11 $d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A	N/A	N/A
12 $d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A	N/A	N/A
13 $d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A	N/A	N/A
14 $d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A	N/A
summary: generation	N/A	N/A	N/A
summary: wall	N/A	N/A	N/A

Baseline: original AmpliCol; candidate: recurrence JIT O2. Each cell shows the baseline generation and wall time, followed by candidate/baseline generation and (wall|native execution) ratios.

UFO-SM recurrence versus AmpliCol full-colour (block 2 of 2)

ID base process	n=4	n=5
1 $d\bar{d} \rightarrow Z + (n-1)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
2 $u\bar{d} \rightarrow W^+ + (n-1)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
3 $d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
4 $u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
5 $d\bar{d} \rightarrow ZZ + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
6 $gg \rightarrow t\bar{t} + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
7 $d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
8 $gg \rightarrow gg + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
9 $d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
10 $d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
11 $d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
12 $d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
13 $d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
14 $d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A
summary: generation	N/A	N/A
summary: wall	N/A	N/A

Baseline: original AmpliCol; candidate: recurrence JIT O2. Each cell shows the baseline generation and wall time, followed by candidate/baseline generation and (wall|native execution) ratios.

5.7 Built-in SM compiled JIT O3 versus recurrence JIT O2 LC

Built-in SM compiled JIT O3 versus recurrence JIT O2 LC (block 1 of 3)

ID base process	n=1	n=2	n=3
1 $d\bar{d} \rightarrow Z + (n-1)g$	19.1 / 31.9 (x0.579) N/A 1.05 / 2.22 (x0.194 x0.0493) N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
2 $u\bar{d} \rightarrow W^+ + (n-1)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
3 $d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
4 $u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
5 $d\bar{d} \rightarrow ZZ + (n-2)g$	N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
6 $gg \rightarrow t\bar{t} + (n-2)g$	N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
7 $d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
8 $gg \rightarrow gg + (n-2)g$	N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
9 $d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A	N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
10 $d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A	N/A	N/A
11 $d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A	N/A	N/A
12 $d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A	N/A	N/A
13 $d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A	N/A	N/A
14 $d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A	N/A
summary: generation	19.1 (x0.579) / N/A	N/A / N/A	N/A / N/A
summary: wall	1.05 (x0.194) / N/A	N/A / N/A	N/A / N/A

Baseline: recurrence JIT O2; candidate: compiled JIT O3. Each cell shows the topology-replay/all-flow-union baseline generation times and wall times, followed by layout-matched candidate/baseline generation and (wall|native execution) ratios.

Built-in SM compiled JIT O3 versus recurrence JIT O2 LC (block 2 of 3)

ID base process	n=4	n=5	n=6
1 $d\bar{d} \rightarrow Z + (n-1)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
2 $u\bar{d} \rightarrow W^+ + (n-1)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
3 $d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
4 $u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
5 $d\bar{d} \rightarrow ZZ + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
6 $gg \rightarrow t\bar{t} + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
7 $d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
8 $gg \rightarrow gg + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
9 $d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
10 $d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
11 $d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
12 $d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
13 $d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
14 $d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
summary: generation	N/A / N/A	N/A / N/A	N/A / N/A
summary: wall	N/A / N/A	N/A / N/A	N/A / N/A

Baseline: recurrence JIT O2; candidate: compiled JIT O3. Each cell shows the topology-replay/all-flow-union baseline generation times and wall times, followed by layout-matched candidate/baseline generation and (wall|native execution) ratios.

Built-in SM compiled JIT O3 versus recurrence JIT O2 LC (block 3 of 3)

ID base process	n=7	n=8	n=9
1 $d\bar{d} \rightarrow Z + (n-1)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
2 $u\bar{d} \rightarrow W^+ + (n-1)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
3 $d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
4 $u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
5 $d\bar{d} \rightarrow ZZ + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
6 $gg \rightarrow t\bar{t} + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A
7 $d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
8 $gg \rightarrow gg + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A
9 $d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
10 $d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
11 $d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
12 $d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
13 $d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A
14 $d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A
summary: generation	N/A / N/A	N/A / N/A	N/A / N/A
summary: wall	N/A / N/A	N/A / N/A	N/A / N/A

Baseline: recurrence JIT O2; candidate: compiled JIT O3. Each cell shows the topology-replay/all-flow-union baseline generation times and wall times, followed by layout-matched candidate/baseline generation and (wall|native execution) ratios.

5.8 Built-in SM compiled JIT O3 versus recurrence JIT O2 NLC

Built-in SM compiled JIT O3 versus recurrence JIT O2 NLC (block 1 of 2)

ID base process	n=1	n=2	n=3
1 $d\bar{d} \rightarrow Z + (n-1)g$	22.1 N/A 0.884 N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
2 $u\bar{d} \rightarrow W^+ + (n-1)g$	N/A N/A N/A N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
3 $d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
4 $u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
5 $d\bar{d} \rightarrow ZZ + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
6 $gg \rightarrow t\bar{t} + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
7 $d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
8 $gg \rightarrow gg + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
9 $d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A	N/A	N/A N/A N/A N/A
10 $d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A	N/A	N/A
11 $d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A	N/A	N/A
12 $d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A	N/A	N/A
13 $d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A	N/A	N/A
14 $d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A	N/A
summary: generation	N/A	N/A	N/A
summary: wall	N/A	N/A	N/A

Baseline: recurrence JIT O2; candidate: compiled JIT O3. Each cell shows the baseline generation and wall time, followed by candidate/baseline generation and (wall|native execution) ratios.

Built-in SM compiled JIT O3 versus recurrence JIT O2 NLC (block 2 of 2)

ID base process	n=4	n=5
1 $d\bar{d} \rightarrow Z + (n-1)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
2 $u\bar{d} \rightarrow W^+ + (n-1)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
3 $d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
4 $u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
5 $d\bar{d} \rightarrow ZZ + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
6 $gg \rightarrow t\bar{t} + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
7 $d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
8 $gg \rightarrow gg + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
9 $d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
10 $d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
11 $d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
12 $d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
13 $d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
14 $d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A
summary: generation	N/A	N/A
summary: wall	N/A	N/A

Baseline: recurrence JIT O2; candidate: compiled JIT O3. Each cell shows the baseline generation and wall time, followed by candidate/baseline generation and (wall|native execution) ratios.

5.9 Built-in SM compiled JIT O3 versus recurrence JIT O2 full-colour

Built-in SM compiled JIT O3 versus recurrence JIT O2 full-colour (block 1 of 2)

ID base process	n=1	n=2	n=3
1 $d\bar{d} \rightarrow Z + (n-1)g$	N/A N/A N/A N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
2 $u\bar{d} \rightarrow W^+ + (n-1)g$	N/A N/A N/A N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
3 $d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
4 $u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
5 $d\bar{d} \rightarrow ZZ + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
6 $gg \rightarrow t\bar{t} + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
7 $d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
8 $gg \rightarrow gg + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
9 $d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A	N/A	N/A N/A N/A N/A
10 $d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A	N/A	N/A
11 $d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A	N/A	N/A
12 $d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A	N/A	N/A
13 $d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A	N/A	N/A
14 $d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A	N/A
summary: generation	N/A	N/A	N/A
summary: wall	N/A	N/A	N/A

Baseline: recurrence JIT O2; candidate: compiled JIT O3. Each cell shows the baseline generation and wall time, followed by candidate/baseline generation and (wall|native execution) ratios.

Built-in SM compiled JIT O3 versus recurrence JIT O2 full-colour (block 2 of 2)

ID base process	n=4	n=5
1 $d\bar{d} \rightarrow Z + (n-1)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
2 $u\bar{d} \rightarrow W^+ + (n-1)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
3 $d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
4 $u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
5 $d\bar{d} \rightarrow ZZ + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
6 $gg \rightarrow t\bar{t} + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
7 $d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
8 $gg \rightarrow gg + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
9 $d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
10 $d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
11 $d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
12 $d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
13 $d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
14 $d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A
summary: generation	N/A	N/A
summary: wall	N/A	N/A

Baseline: recurrence JIT O2; candidate: compiled JIT O3. Each cell shows the baseline generation and wall time, followed by candidate/baseline generation and (wall|native execution) ratios.

5.10 Built-in SM eager-DAG JIT O2 versus recurrence JIT O2 LC

Built-in SM eager-DAG JIT O2 versus recurrence JIT O2 LC (block 1 of 3)

ID base process	n=1	n=2	n=3
1 $d\bar{d} \rightarrow Z + (n-1)g$	19.1 / 31.9 (x0.702) N/A 1.05 / 2.22 (x2.62 x1.43) N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
2 $u\bar{d} \rightarrow W^+ + (n-1)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
3 $d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
4 $u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
5 $d\bar{d} \rightarrow ZZ + (n-2)g$	N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
6 $gg \rightarrow t\bar{t} + (n-2)g$	N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
7 $d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
8 $gg \rightarrow gg + (n-2)g$	N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
9 $d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A	N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
10 $d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A	N/A	N/A
11 $d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A	N/A	N/A
12 $d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A	N/A	N/A
13 $d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A	N/A	N/A
14 $d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A	N/A
summary: generation	19.1 (x0.702) / N/A	N/A / N/A	N/A / N/A
summary: wall	1.05 (x2.62) / N/A	N/A / N/A	N/A / N/A

Baseline: recurrence JIT O2; candidate: eager-DAG JIT O2. Each cell shows the topology-replay/all-flow-union baseline generation times and wall times, followed by layout-matched candidate/baseline generation and (wall|native execution) ratios.

Built-in SM eager-DAG JIT O2 versus recurrence JIT O2 LC (block 2 of 3)

ID base process	n=4	n=5	n=6
1 $d\bar{d} \rightarrow Z + (n-1)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
2 $u\bar{d} \rightarrow W^+ + (n-1)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
3 $d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
4 $u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
5 $d\bar{d} \rightarrow ZZ + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
6 $gg \rightarrow t\bar{t} + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
7 $d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
8 $gg \rightarrow gg + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
9 $d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
10 $d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
11 $d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
12 $d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
13 $d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
14 $d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
summary: generation	N/A / N/A	N/A / N/A	N/A / N/A
summary: wall	N/A / N/A	N/A / N/A	N/A / N/A

Baseline: recurrence JIT O2; candidate: eager-DAG JIT O2. Each cell shows the topology-replay/all-flow-union baseline generation times and wall times, followed by layout-matched candidate/baseline generation and (wall|native execution) ratios.

Built-in SM eager-DAG JIT O2 versus recurrence JIT O2 LC (block 3 of 3)

ID base process	n=7	n=8	n=9
1 $d\bar{d} \rightarrow Z + (n-1)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
2 $u\bar{d} \rightarrow W^+ + (n-1)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
3 $d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
4 $u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
5 $d\bar{d} \rightarrow ZZ + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
6 $gg \rightarrow t\bar{t} + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A
7 $d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
8 $gg \rightarrow gg + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A
9 $d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
10 $d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
11 $d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
12 $d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
13 $d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A
14 $d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A
summary: generation	N/A / N/A	N/A / N/A	N/A / N/A
summary: wall	N/A / N/A	N/A / N/A	N/A / N/A

Baseline: recurrence JIT O2; candidate: eager-DAG JIT O2. Each cell shows the topology-replay/all-flow-union baseline generation times and wall times, followed by layout-matched candidate/baseline generation and (wall|native execution) ratios.

5.11 Built-in SM eager-DAG JIT O2 versus recurrence JIT O2 NLC

Built-in SM eager-DAG JIT O2 versus recurrence JIT O2 NLC (block 1 of 2)

ID base process	n=1	n=2	n=3
1 $d\bar{d} \rightarrow Z + (n-1)g$	22.1 N/A 0.884 N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
2 $u\bar{d} \rightarrow W^+ + (n-1)g$	N/A N/A N/A N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
3 $d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
4 $u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
5 $d\bar{d} \rightarrow ZZ + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
6 $gg \rightarrow t\bar{t} + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
7 $d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
8 $gg \rightarrow gg + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
9 $d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A	N/A	N/A N/A N/A N/A
10 $d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A	N/A	N/A
11 $d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A	N/A	N/A
12 $d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A	N/A	N/A
13 $d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A	N/A	N/A
14 $d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A	N/A
summary: generation	N/A	N/A	N/A
summary: wall	N/A	N/A	N/A

Baseline: recurrence JIT O2; candidate: eager-DAG JIT O2. Each cell shows the baseline generation and wall time, followed by candidate/baseline generation and (wall|native execution) ratios.

Built-in SM eager-DAG JIT O2 versus recurrence JIT O2 NLC (block 2 of 2)

ID base process	n=4	n=5
1 $d\bar{d} \rightarrow Z + (n-1)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
2 $u\bar{d} \rightarrow W^+ + (n-1)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
3 $d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
4 $u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
5 $d\bar{d} \rightarrow ZZ + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
6 $gg \rightarrow t\bar{t} + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
7 $d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
8 $gg \rightarrow gg + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
9 $d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
10 $d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
11 $d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
12 $d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
13 $d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
14 $d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A
summary: generation	N/A	N/A
summary: wall	N/A	N/A

Baseline: recurrence JIT O2; candidate: eager-DAG JIT O2. Each cell shows the baseline generation and wall time, followed by candidate/baseline generation and (wall|native execution) ratios.

5.12 Built-in SM eager-DAG JIT O2 versus recurrence JIT O2 full-colour

Built-in SM eager-DAG JIT O2 versus recurrence JIT O2 full-colour (block 1 of 2)

ID base process	n=1	n=2	n=3
1 $d\bar{d} \rightarrow Z + (n-1)g$	N/A N/A N/A N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
2 $u\bar{d} \rightarrow W^+ + (n-1)g$	N/A N/A N/A N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
3 $d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
4 $u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
5 $d\bar{d} \rightarrow ZZ + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
6 $gg \rightarrow t\bar{t} + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
7 $d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
8 $gg \rightarrow gg + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
9 $d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A	N/A	N/A N/A N/A N/A
10 $d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A	N/A	N/A
11 $d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A	N/A	N/A
12 $d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A	N/A	N/A
13 $d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A	N/A	N/A
14 $d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A	N/A
summary: generation	N/A	N/A	N/A
summary: wall	N/A	N/A	N/A

Baseline: recurrence JIT O2; candidate: eager-DAG JIT O2. Each cell shows the baseline generation and wall time, followed by candidate/baseline generation and (wall|native execution) ratios.

Built-in SM eager-DAG JIT O2 versus recurrence JIT O2 full-colour (block 2 of 2)

ID base process	n=4	n=5
1 $d\bar{d} \rightarrow Z + (n-1)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
2 $u\bar{d} \rightarrow W^+ + (n-1)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
3 $d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
4 $u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
5 $d\bar{d} \rightarrow ZZ + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
6 $gg \rightarrow t\bar{t} + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
7 $d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
8 $gg \rightarrow gg + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
9 $d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
10 $d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
11 $d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
12 $d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
13 $d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
14 $d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A
summary: generation	N/A	N/A
summary: wall	N/A	N/A

Baseline: recurrence JIT O2; candidate: eager-DAG JIT O2. Each cell shows the baseline generation and wall time, followed by candidate/baseline generation and (wall|native execution) ratios.

6 Z-plus-Jets Evaluator Ladders

The dedicated $d\bar{d} \rightarrow Z + (n-1)g$ ladders compare the independent reference with compiled JIT O1, compiled JIT O3, ASM O3, C++ O3, eager-DAG JIT O2, and recurrence JIT O2 evaluator configurations. They retain final-state multiplicities $n = 1, \dots, 9$ for both the built-in model and the UFO-SM result set. Each row states its evaluator backend and optimization level explicitly; all other workload settings follow the benchmark contract above.

The eager and recurrence rows start from the corresponding portable JIT O2 prepared model. Each creates two complete process artifacts: topology replay for selected-flow/helicity-sum and all-flow union for all-flows/single-helicity. Each workload block reports its own process generation time and applies its flow or helicity selector at runtime. Prepared-model creation is excluded from both process-generation entries and retained as separate provenance. Every displayed Z-ladder ratio continues to use original `AmpliCol` as its denominator; shared-core pointwise checks additionally compare the `pyAmpliCol` modes with the matching validated recurrence or compiled result at the stricter tolerance.

Runtime comparisons between these backends also include their vectorization strategy. The `SymJIT` application evaluates a batch with SIMD across phase-space points, whereas generated ASM and C++ payloads are scalar and `Rusticol` invokes them point by point. Their ratios therefore measure both symbolic/code-generation quality and this SIMD advantage; they must not be interpreted as a comparison of otherwise identical vectorized kernels.

Worked $d\bar{d} \rightarrow Zg$ example

For $n = 2$, the explicit process is

$$d(p_1) \bar{d}(p_2) \rightarrow Z(p_3) g(p_4).$$

The report labels source particles in process order. The selected leading-colour reference order for the selected-flow timing is therefore $(2, 4, 1, 3)$: the colour line enters through $\bar{d}$, traverses the gluon, and exits through d , while the colour-singlet Z remains in the public ordering only to keep source labels unambiguous. Internally, the colour-sector match is the coloured word $(2, 4, 1)$.

`pyAmpliCol` builds a shared-current DAG for the selected process. Source nodes carry particle, momentum, helicity, flavour, and colour-flow labels; composite nodes are keyed by the full current identity and the external-label mask. When two diagrams need the same current, the DAG contains one node and downstream amplitude roots reuse its value. Both LC report artifacts retain all physical flows and helicities. The topology-replay artifact selects $(2, 4, 1)$ at runtime and sums source helicities; the all-flow-union artifact selects one source-helicity assignment at runtime and sums every LC flow. Each generated artifact serializes its own stages and evaluator payloads; `Rusticol` loads it, executes `Runtime.evaluate()` inside the native core, and `BenchmarkRunner` records both the native repeated wall time and the sum of evaluator-call timings.

6.1 Built-in SM dedicated $d\bar{d} \rightarrow Z + \text{gluon}$ performance

Built-in SM $d\bar{d} \rightarrow Z + \text{gluon ladder}$ (block 1 of 3)

n setup	selected flow, helicity sum			all flows, single helicity		
	gen [s]	wall [us/pt]	exec [us/pt]	gen [s]	wall [us/pt]	exec [us/pt]
1 AmpliCol reference	15.4	0.169	N/A	0.00034	0.249	N/A
1 compiled JIT O1	N/A	N/A	N/A	N/A	N/A	N/A
1 compiled ASM O3	N/A	N/A	N/A	N/A	N/A	N/A
1 compiled C++ O3	N/A	N/A	N/A	N/A	N/A	N/A
1 compiled JIT O3	N/A	N/A	N/A	N/A	N/A	N/A
1 eager-DAG JIT O2	N/A	N/A	N/A	N/A	N/A	N/A
1 recurrence JIT O2	N/A	N/A	N/A	N/A	N/A	N/A
2 AmpliCol reference	N/A	N/A	N/A	N/A	N/A	N/A
2 compiled JIT O1	N/A	N/A	N/A	N/A	N/A	N/A
2 compiled ASM O3	N/A	N/A	N/A	N/A	N/A	N/A
2 compiled C++ O3	N/A	N/A	N/A	N/A	N/A	N/A
2 compiled JIT O3	N/A	N/A	N/A	N/A	N/A	N/A
2 eager-DAG JIT O2	N/A	N/A	N/A	N/A	N/A	N/A
2 recurrence JIT O2	N/A	N/A	N/A	N/A	N/A	N/A
3 AmpliCol reference	N/A	N/A	N/A	N/A	N/A	N/A
3 compiled JIT O1	N/A	N/A	N/A	N/A	N/A	N/A
3 compiled ASM O3	N/A	N/A	N/A	N/A	N/A	N/A
3 compiled C++ O3	N/A	N/A	N/A	N/A	N/A	N/A
3 compiled JIT O3	N/A	N/A	N/A	N/A	N/A	N/A
3 eager-DAG JIT O2	N/A	N/A	N/A	N/A	N/A	N/A
3 recurrence JIT O2	N/A	N/A	N/A	N/A	N/A	N/A

Original AmpliCol is the denominator. Each pyAmpliCol row reports separate topology-replay and all-flow-union generation and runtime measurements. Parenthesized values are candidate/reference ratios.

Built-in SM $d\bar{d} \rightarrow Z + \text{gluon ladder}$ (block 2 of 3)

n setup	selected flow, helicity sum			all flows, single helicity		
	gen [s]	wall [us/pt]	exec [us/pt]	gen [s]	wall [us/pt]	exec [us/pt]
4 AmpliCol reference	N/A	N/A	N/A	N/A	N/A	N/A
4 compiled JIT O1	N/A	N/A	N/A	N/A	N/A	N/A
4 compiled ASM O3	N/A	N/A	N/A	N/A	N/A	N/A
4 compiled C++ O3	N/A	N/A	N/A	N/A	N/A	N/A
4 compiled JIT O3	N/A	N/A	N/A	N/A	N/A	N/A
4 eager-DAG JIT O2	N/A	N/A	N/A	N/A	N/A	N/A
4 recurrence JIT O2	N/A	N/A	N/A	N/A	N/A	N/A
5 AmpliCol reference	N/A	N/A	N/A	N/A	N/A	N/A
5 compiled JIT O1	N/A	N/A	N/A	N/A	N/A	N/A
5 compiled ASM O3	N/A	N/A	N/A	N/A	N/A	N/A
5 compiled C++ O3	N/A	N/A	N/A	N/A	N/A	N/A
5 compiled JIT O3	N/A	N/A	N/A	N/A	N/A	N/A
5 eager-DAG JIT O2	N/A	N/A	N/A	N/A	N/A	N/A
5 recurrence JIT O2	N/A	N/A	N/A	N/A	N/A	N/A
6 AmpliCol reference	N/A	N/A	N/A	N/A	N/A	N/A
6 compiled JIT O1	N/A	N/A	N/A	N/A	N/A	N/A
6 compiled ASM O3	N/A	N/A	N/A	N/A	N/A	N/A
6 compiled C++ O3	N/A	N/A	N/A	N/A	N/A	N/A
6 compiled JIT O3	N/A	N/A	N/A	N/A	N/A	N/A
6 eager-DAG JIT O2	N/A	N/A	N/A	N/A	N/A	N/A
6 recurrence JIT O2	N/A	N/A	N/A	N/A	N/A	N/A

Original AmpliCol is the denominator. Each pyAmpliCol row reports separate topology-replay and all-flow-union generation and runtime measurements. Parenthesized values are candidate/reference ratios.

Built-in SM $d\bar{d} \rightarrow Z + \text{gluon ladder}$ (block 3 of 3)

n setup	selected flow, helicity sum			all flows, single helicity		
	gen [s]	wall [us/pt]	exec [us/pt]	gen [s]	wall [us/pt]	exec [us/pt]
7 AmpliCol reference	N/A	N/A	N/A	N/A	N/A	N/A
7 compiled JIT O1	N/A	N/A	N/A	N/A	N/A	N/A
7 compiled ASM O3	N/A	N/A	N/A	N/A	N/A	N/A
7 compiled C++ O3	N/A	N/A	N/A	N/A	N/A	N/A
7 compiled JIT O3	N/A	N/A	N/A	N/A	N/A	N/A
7 eager-DAG JIT O2	N/A	N/A	N/A	N/A	N/A	N/A
7 recurrence JIT O2	N/A	N/A	N/A	N/A	N/A	N/A
8 AmpliCol reference	N/A	N/A	N/A	N/A	N/A	N/A
8 compiled JIT O1	N/A	N/A	N/A	N/A	N/A	N/A
8 compiled ASM O3	N/A	N/A	N/A	N/A	N/A	N/A
8 compiled C++ O3	N/A	N/A	N/A	N/A	N/A	N/A
8 compiled JIT O3	N/A	N/A	N/A	N/A	N/A	N/A
8 eager-DAG JIT O2	N/A	N/A	N/A	N/A	N/A	N/A
8 recurrence JIT O2	N/A	N/A	N/A	N/A	N/A	N/A
9 AmpliCol reference	N/A	N/A	N/A	N/A	N/A	N/A
9 compiled JIT O1	N/A	N/A	N/A	N/A	N/A	N/A
9 compiled ASM O3	N/A	N/A	N/A	N/A	N/A	N/A
9 compiled C++ O3	N/A	N/A	N/A	N/A	N/A	N/A
9 compiled JIT O3	N/A	N/A	N/A	N/A	N/A	N/A
9 eager-DAG JIT O2	N/A	N/A	N/A	N/A	N/A	N/A
9 recurrence JIT O2	N/A	N/A	N/A	N/A	N/A	N/A

Original AmpliCol is the denominator. Each pyAmpliCol row reports separate topology-replay and all-flow-union generation and runtime measurements. Parenthesized values are candidate/reference ratios.

6.2 UFO-SM dedicated $d\bar{d} \rightarrow Z + \text{gluon}$ performance

UFO-SM $d\bar{d} \rightarrow Z + \text{gluon ladder}$ (block 1 of 3)

n setup	selected flow, helicity sum			all flows, single helicity		
	gen [s]	wall [us/pt]	exec [us/pt]	gen [s]	wall [us/pt]	exec [us/pt]
1 AmpliCol reference	15.4	0.169	N/A	0.00034	0.249	N/A
1 compiled JIT O1	N/A	N/A	N/A	N/A	N/A	N/A
1 compiled ASM O3	N/A	N/A	N/A	N/A	N/A	N/A
1 compiled C++ O3	N/A	N/A	N/A	N/A	N/A	N/A
1 compiled JIT O3	N/A	N/A	N/A	N/A	N/A	N/A
1 eager-DAG JIT O2	N/A	N/A	N/A	N/A	N/A	N/A
1 recurrence JIT O2	N/A	N/A	N/A	N/A	N/A	N/A
2 AmpliCol reference	N/A	N/A	N/A	N/A	N/A	N/A
2 compiled JIT O1	N/A	N/A	N/A	N/A	N/A	N/A
2 compiled ASM O3	N/A	N/A	N/A	N/A	N/A	N/A
2 compiled C++ O3	N/A	N/A	N/A	N/A	N/A	N/A
2 compiled JIT O3	N/A	N/A	N/A	N/A	N/A	N/A
2 eager-DAG JIT O2	N/A	N/A	N/A	N/A	N/A	N/A
2 recurrence JIT O2	N/A	N/A	N/A	N/A	N/A	N/A
3 AmpliCol reference	N/A	N/A	N/A	N/A	N/A	N/A
3 compiled JIT O1	N/A	N/A	N/A	N/A	N/A	N/A
3 compiled ASM O3	N/A	N/A	N/A	N/A	N/A	N/A
3 compiled C++ O3	N/A	N/A	N/A	N/A	N/A	N/A
3 compiled JIT O3	N/A	N/A	N/A	N/A	N/A	N/A
3 eager-DAG JIT O2	N/A	N/A	N/A	N/A	N/A	N/A
3 recurrence JIT O2	N/A	N/A	N/A	N/A	N/A	N/A

Original AmpliCol is the denominator. Each pyAmpliCol row reports separate topology-replay and all-flow-union generation and runtime measurements. Parenthesized values are candidate/reference ratios.

UFO-SM $d\bar{d} \rightarrow Z + \text{gluon ladder}$ (block 2 of 3)

n setup	selected flow, helicity sum			all flows, single helicity		
	gen [s]	wall [us/pt]	exec [us/pt]	gen [s]	wall [us/pt]	exec [us/pt]
4 AmpliCol reference	N/A	N/A	N/A	N/A	N/A	N/A
4 compiled JIT O1	N/A	N/A	N/A	N/A	N/A	N/A
4 compiled ASM O3	N/A	N/A	N/A	N/A	N/A	N/A
4 compiled C++ O3	N/A	N/A	N/A	N/A	N/A	N/A
4 compiled JIT O3	N/A	N/A	N/A	N/A	N/A	N/A
4 eager-DAG JIT O2	N/A	N/A	N/A	N/A	N/A	N/A
4 recurrence JIT O2	N/A	N/A	N/A	N/A	N/A	N/A
5 AmpliCol reference	N/A	N/A	N/A	N/A	N/A	N/A
5 compiled JIT O1	N/A	N/A	N/A	N/A	N/A	N/A
5 compiled ASM O3	N/A	N/A	N/A	N/A	N/A	N/A
5 compiled C++ O3	N/A	N/A	N/A	N/A	N/A	N/A
5 compiled JIT O3	N/A	N/A	N/A	N/A	N/A	N/A
5 eager-DAG JIT O2	N/A	N/A	N/A	N/A	N/A	N/A
5 recurrence JIT O2	N/A	N/A	N/A	N/A	N/A	N/A
6 AmpliCol reference	N/A	N/A	N/A	N/A	N/A	N/A
6 compiled JIT O1	N/A	N/A	N/A	N/A	N/A	N/A
6 compiled ASM O3	N/A	N/A	N/A	N/A	N/A	N/A
6 compiled C++ O3	N/A	N/A	N/A	N/A	N/A	N/A
6 compiled JIT O3	N/A	N/A	N/A	N/A	N/A	N/A
6 eager-DAG JIT O2	N/A	N/A	N/A	N/A	N/A	N/A
6 recurrence JIT O2	N/A	N/A	N/A	N/A	N/A	N/A

Original AmpliCol is the denominator. Each pyAmpliCol row reports separate topology-replay and all-flow-union generation and runtime measurements. Parenthesized values are candidate/reference ratios.

UFO-SM $d\bar{d} \rightarrow Z + \text{gluon ladder}$ (block 3 of 3)

n setup	selected flow, helicity sum			all flows, single helicity		
	gen [s]	wall [us/pt]	exec [us/pt]	gen [s]	wall [us/pt]	exec [us/pt]
7 AmpliCol reference	N/A	N/A	N/A	N/A	N/A	N/A
7 compiled JIT O1	N/A	N/A	N/A	N/A	N/A	N/A
7 compiled ASM O3	N/A	N/A	N/A	N/A	N/A	N/A
7 compiled C++ O3	N/A	N/A	N/A	N/A	N/A	N/A
7 compiled JIT O3	N/A	N/A	N/A	N/A	N/A	N/A
7 eager-DAG JIT O2	N/A	N/A	N/A	N/A	N/A	N/A
7 recurrence JIT O2	N/A	N/A	N/A	N/A	N/A	N/A
8 AmpliCol reference	N/A	N/A	N/A	N/A	N/A	N/A
8 compiled JIT O1	N/A	N/A	N/A	N/A	N/A	N/A
8 compiled ASM O3	N/A	N/A	N/A	N/A	N/A	N/A
8 compiled C++ O3	N/A	N/A	N/A	N/A	N/A	N/A
8 compiled JIT O3	N/A	N/A	N/A	N/A	N/A	N/A
8 eager-DAG JIT O2	N/A	N/A	N/A	N/A	N/A	N/A
8 recurrence JIT O2	N/A	N/A	N/A	N/A	N/A	N/A
9 AmpliCol reference	N/A	N/A	N/A	N/A	N/A	N/A
9 compiled JIT O1	N/A	N/A	N/A	N/A	N/A	N/A
9 compiled ASM O3	N/A	N/A	N/A	N/A	N/A	N/A
9 compiled C++ O3	N/A	N/A	N/A	N/A	N/A	N/A
9 compiled JIT O3	N/A	N/A	N/A	N/A	N/A	N/A
9 eager-DAG JIT O2	N/A	N/A	N/A	N/A	N/A	N/A
9 recurrence JIT O2	N/A	N/A	N/A	N/A	N/A	N/A

Original AmpliCol is the denominator. Each pyAmpliCol row reports separate topology-replay and all-flow-union generation and runtime measurements. Parenthesized values are candidate/reference ratios.

Each LC workload block reports its own process generation in seconds and Rusticol-core wall and native execution-submetric runtime in microseconds per point: topology replay for selected-flow/helicity-sum, and all-flow union for all-flows/single-helicity. Compiled rows use JIT O1/O3 or the labelled scalar backend; eager-DAG and recurrence rows use portable JIT O2 prepared models. Parenthesized multipliers compare each pyAmpliCol row with original AmpliCol.

7 Colour-Singlet Model Ladders

The scalar-contact ladder retains $n = 2, \dots, 8$ final scalars. It tests generic high-valence contact decomposition, identical-particle normalization, and source parity without Standard-Model-specific kernels. The scalar-gravity ladder retains $n = 2, \dots, 4$ final gravitons and exercises spin-two wavefunctions, tensor propagators, and momentum-dependent vertices. Since all external particles in these fixtures are colour singlets, LC, NLC, and full colour reduce to the same unit colour contraction.

For these ladders, the relative-difference row is $|M_{f64} - M_{hp}| / \max(|M_{hp}|, 10^{-300})$, where M_{hp} is the same point re-evaluated through the higher-precision diagnostic path. It is a numerical stability check, not a performance ratio.

7.1 Scalar-contact ladder

metric	scalar_0 scalar_0 > n*scalar_0						
	n=2	n=3	n=4	n=5	n=6	n=7	n=8
generation [s]	N/A	N/A	N/A	N/A	N/A	N/A	N/A
wall [μ s/pt]	N/A	N/A	N/A	N/A	N/A	N/A	N/A
execution [μ s/pt]	N/A	N/A	N/A	N/A	N/A	N/A	N/A
matrix element	N/A	N/A	N/A	N/A	N/A	N/A	N/A
relative diff. vs hp	N/A	N/A	N/A	N/A	N/A	N/A	N/A

7.2 Scalar-gravity ladder

metric	scalar_0 scalar_0 > n*graviton		
	n=2	n=3	n=4
generation [s]	N/A	N/A	N/A
wall [μ s/pt]	N/A	N/A	N/A
execution [μ s/pt]	N/A	N/A	N/A
matrix element	N/A	N/A	N/A
relative diff. vs hp	N/A	N/A	N/A

8 Interpretation

This report establishes coverage, methodology, and the measured result set. Performance comparisons are meaningful only for a reviewed baseline produced from the same effective configuration on named hardware. Readers should treat every N/A entry as unavailable and should not interpolate, copy, or estimate a value from neighbouring multiplicities or model profiles.

The retained measurement records under `docs/results/` are authoritative for table completeness and provenance. The TeX inputs and this PDF are their publication presentation.

9 Worked $d\bar{d} \rightarrow Zgg$ Example

This section follows one process from its user-facing notation to the amplitude roots generated by `pyAmpliCol`. It is deliberately concrete: the counts and labels below are those produced by the built-in Standard Model for complete leading-colour (LC) coverage, not a schematic diagram count. The same process is also contained in the UFO-SM validation set.

9.1 External legs and conventions

The physical process is

$$d(p_1) \bar{d}(p_2) \longrightarrow Z(p_3) g(p_4) g(p_5), \quad p_1 + p_2 = p_3 + p_4 + p_5. \quad (6)$$

The integers $1, \dots, 5$ are *source labels*. They are stable labels of external particles in process order; they are neither current IDs nor colour-sector IDs. The DAG compiler crosses incoming particles once and then uses the all-outgoing momenta

$$q_1 = -p_1, \quad q_2 = -p_2, \quad q_i = p_i \ (i = 3, 4, 5), \quad \sum_{i=1}^5 q_i = 0. \quad (7)$$

Runtime clients nevertheless supply momenta in the physical order and sign convention of Eq. (6); Rusticol performs the two incoming sign changes when it fills momentum slots.

label	physical leg	DAG particle	DAG momentum	states	colour role
1	incoming d	outgoing $\bar{d}$	$q_1 = -p_1$	2	$\bar{\mathbf{3}}$, colour-line end
2	incoming $\bar{d}$	outgoing d	$q_2 = -p_2$	2	$\mathbf{3}$, colour-line start
3	outgoing Z	outgoing Z	$q_3 = p_3$	3	singlet
4	outgoing g	outgoing g	$q_4 = p_4$	2	adjoint insertion
5	outgoing g	outgoing g	$q_5 = p_5$	2	adjoint insertion

There are $2 \times 2 \times 3 \times 2 \times 2 = 48$ physical helicity configurations. The physics metadata retains all 48 stable helicity IDs, for example `h:+1,-1,+0,-1,+1`. Twenty-four configurations are proven structural zeros by the chiral source and kernel constraints and are returned as explicit zeros by resolved evaluation; only the nonzero representatives need amplitude roots.

At LC the singlet label 3 is absent from the coloured word. Complete coverage has the two physical flows

$$c_0 = (2, 4, 5, 1), \quad c_1 = (2, 5, 4, 1), \quad (8)$$

published as the stable runtime IDs `flow:2,4,5,1` and `flow:2,5,4,1`. A user selects these strings. The small integer sector used while building a DAG is artifact-local implementation data and must not be used as a public flow identifier.

9.2 From diagrams to reusable currents

For a nonempty source subset I , define

$$q_I = \sum_{i \in I} q_i, \quad m(I) = \sum_{i \in I} 2^{i-1}. \quad (9)$$

Thus $m(\{2, 4, 5\}) = 26$. A source wavefunction is a one-leg current $J_i^a = W_{t_i}^a(q_i, h_i)$, where W denotes the appropriate spinor, polarization vector, or scalar wavefunction. Composite currents obey the binary recursion

$$J_{t,\Omega}^a(I) = P_t^a{}_b(q_I) \sum_{\substack{I=A \dot{\cup} B \\ A, B \neq \emptyset}} \sum_{v: t_A t_B \rightarrow t} \kappa_v V_v^b{}_{cd}(q_A, q_B) J_{t_A, \Omega_A}^c(A) J_{t_B, \Omega_B}^d(B). \quad (10)$$

Here t is the particle state and Ω is the complete current identity described in Sec. 10. The sum contains only model, quantum-number, perturbative-order, and colour-order admissible combinations. The propagator P_t belongs to the output value slot; an unpropagated form is retained as well when a later closure needs it.

For one fixed source-state ancestry, representative terms along the quark line are

$$J_d(2, 5) = P_d V_{dgd} [J_d(2), J_g(5)], \quad (11)$$

$$J_g(4, 5) = P_g V_{ggg} [J_g(4), J_g(5)], \quad (12)$$

$$J_d(2, 4, 5) = P_d \left(V_{dgd} [J_d(2, 5), J_g(4)] + V_{dgd} [J_d(2, 4), J_g(5)] + V_{dgd} [J_d(2), J_g(4, 5)] \right), \quad (13)$$

$$J_d(2, 3, 4, 5) = P_d \sum_{A \dot{\cup} B = \{2, 3, 4, 5\}} V[J(A), J(B)]. \quad (14)$$

Equation (14) includes all admissible positions of the Z emission and both gluon traversals. The notation suppresses chirality, helicity ancestry, couplings, and ordered-label data; those fields are never suppressed in the actual key. In particular, $J_d(2, 5, 4)$ and $J_d(2, 4, 5)$ are distinct ordered currents even though they carry the same unordered subset and momentum.

The final closure joins the endpoint current to source label 1:

$$\mathcal{A}_{h,c}^{(r)} = C_{aa}^{(r)} J_d^a(1) J_d^a(2, 3, 4, 5), \quad \mathcal{A}_{h,c} = \sum_{r \in G(h,c)} \mathcal{A}_{h,c}^{(r)}. \quad (15)$$

The contraction $C^{(r)}$, root coupling, colour weight, and coherent-group identifier are model-owned records, rather than special cases in Rusticol.

9.3 The concrete graph

The current compiler creates the following deterministic topology. IDs are dense diagnostic indices in this artifact, not stable public selectors.

layer	curr.	attach.	kernel eval.	representative content
sources	11	0	0	two Weyl states on each quark leg, three Z states, and two states on each gluon; current IDs 0–10
$ I = 2$	18	18	18	$J_d(2, 3)$, $J_d(2, 4)$, $J_d(2, 5)$, and $J_g(4, 5)$; current IDs 11–28
$ I = 3$	40	80	72	mixed Zg quark currents and the two ordered $J_d(2, 4, 5)$ families; current IDs 29–68
$ I = 4$	48	144	120	endpoint currents with stored orders $(2, 5, 4, 3)$ and $(2, 4, 5, 3)$; current IDs 69–116
amplitude	—	48 roots	48 outputs	24 nonzero helicities for each physical colour flow
total	117	242	210	no auxiliary current is needed for this process

An *attachment* says that one interaction contributes to one target current. It need not imply a fresh kernel call. For example, the source quark current combined with $J_g(4, 5)$ can feed both ordered three-leg targets. The two attachments share one verified `evaluation_group_id`; a sign or scalar `evaluation_factor` is applied at each target. This is why the three- and four-leg layers contain 224 attachments but only 192 distinct kernel evaluations. The resulting sharing is stronger than merely caching a list of Feynman diagrams.

9.4 Coverage, execution layout, and reproduction

Normal public generation materializes both flows and complete helicity metadata. Runtime selection then uses the stable physical IDs. For the report’s selected-flow/helicity-sum workload, the topology-replay layout maps the source-label order $(2, 5, 4, 1, 3)$ to the coloured traversal $(2, 5, 4, 1)$; the singlet 3 remains source-order provenance. The all-flow-union layout retains the same two public flows but constructs their shared recurrence directly. Section 11 gives the two layout contracts. Neither report artifact fixes this flow or a helicity during generation.

A deliberately selected-coverage diagnostic artifact can restrict this process to one flow. Such an artifact has 65 currents, 90 interactions, and 24 roots, but those internal counts and its artifact-local sector ID are not substitutes for the public flow ID and are not used for reusable report timings.

name space	example	meaning
source label	4	fourth particle in the user process
all-outgoing leg	$g(q_4)$	crossed DAG input with $q_4 = p_4$
internal sector ID	0	artifact-local generation index
physical flow ID	flow:2,5,4,1	stable runtime selector
current ID	69	dense slot for one complete current identity
value slot	propagated current 69	runtime storage variant and component range
root/output	one (h, c) contribution	complex amplitude before squaring

For the built-in compatibility model, Rusticol applies

$$|\mathcal{M}|^2 = \mathcal{N} \sum_{h,c} w_{h,c} \left| \sum_{r \in G(h,c)} \mathcal{A}_{h,c}^{(r)} \right|^2, \quad \mathcal{N} = \frac{27 (4\pi\alpha_s)^2 (8\pi\alpha_{EW})}{36 \times 2}. \quad (16)$$

The denominator averages the two incoming colour/spin state spaces (6×6); the final factor $2 = 2!$ accounts for identical gluons. The physics manifest records the factors, coupling powers, coherent groups, and symmetry weights separately, so changing runtime model parameters refreshes the normalization without rebuilding evaluator code.

The installed examples provide a checkout-independent public route:

```
pyamplicol examples copy ./pyamplicol-examples
cd pyamplicol-examples
pyamplicol generate_pp_zjj_from_ufo_sm.toml
pyamplicol inspect artifacts/pp_zjj --process 'd d~ > z g g'
pyamplicol profile artifacts/pp_zjj --process 'd d~ > z g g' \
  --target-runtime 1 --batch-size 128
```

The generated artifact contains a process set; the exact expression is therefore the clearest selector. In Python, the same complete artifact exposes both the summed and resolved contracts:

```
from pyamplicol import Runtime

runtime = Runtime.load("artifacts/pp_zjj", process="d d~ > z g g")
print(runtime.physics.color_flow_ids)
total = runtime.evaluate(momenta)
resolved = runtime.evaluate_resolved(
    momenta, color_flows=["flow:2,5,4,1"]
)
```

Public TOML and CLI generation retain complete flow and helicity coverage for this workload. Deliberately selected coverage is an expert diagnostic and is not used for the report measurements. Both coverage forms use the same public process metadata and numerical reduction.

10 The $d\bar{d} \rightarrow Zgg$ DAG and Rusticol

The previous section described the physics graph. This section gives the exact mapping from that graph to compiled multi-output evaluators and then to the Rusticol runtime. The main invariant is simple: generation owns physics identity and symbolic lowering; Rusticol owns a validated numerical schedule of slots, stages, roots, and reductions.

10.1 What identifies a current?

The immutable `CurrentIndex` key is defined in `src/pyamplicol/generation/dag_types.py`. Two contributions share a current if and only if every field in

$$\Omega(J) = (t, m_I, I, H, \chi, s, F, Q, C, m_p, O, a, \pi_I) \quad (17)$$

is equal. There is no process-family or diagram label in this key.

field	example	role
<code>particle_id</code> t	1	off-shell particle carried by the current
<code>external_mask</code> m_I	26	bit mask of source subset $I = \{2, 4, 5\}$
<code>external_labels</code> I	(2, 4, 5)	sorted, unordered source-label set
<code>helicity_ancestry</code> H	source-state bit set	exact one-leg state choices feeding the current
<code>chirality</code> χ	+1 or -1	Weyl-flow branch when applicable
<code>spin_state</code> s	scalar or tuple	local spin/polarization state beyond chirality
<code>flavour_flow</code> F	model tuple	conserved flavour-line identity
<code>quantum_number_flow</code> Q	name/expression pairs	exact model-derived additive charges
<code>color_state</code> C	LC state	accuracy, internal sector, line groups, and basis keys
<code>momentum_mask</code> m_p	26	momentum slot used by kernels and propagator
<code>coupling_orders</code> O	QCD=2	accumulated perturbative order envelope
<code>auxiliary_kind</code> a	None	proof-backed contact-decomposition species, if present
<code>ordered_external_labels</code>	(2, 5, 4)	colour-sensitive traversal within the same subset
π_I		

Both the unordered labels and their order are intentional: the former fixes the momentum and subset, while the latter prevents two colour-ordered currents from being merged incorrectly. Conversely, exact equality permits reuse across every diagram and amplitude root that reaches the same state. The `_CurrentTable.add_or_get()` operation in `src/pyamplicol/generation/dag_table.py` is the sole identity-deduplication point. The resulting `current_id` is only the dense array index of that canonical node.

LC generation adds a second, complementary reuse mechanism. Internal currents are built in a shared-ordering state and their ordered labels and basis keys record which flow segments they can serve. Physical sector IDs are restored only when a colour line closes into an amplitude root. The two roots in Eq. (8) therefore share sources, electroweak subcurrents, the $J_g(4, 5)$ family, and every other exactly equal current, instead of cloning a complete recursion per flow.

Each one-leg state receives its own bit when sources are constructed; composing currents takes the bitwise OR. Thus H identifies concrete source-state choices, rather than merely the set of external labels.

Finally, `evaluation_group_id` addresses reuse that current identity alone cannot express. If one verified kernel value contributes, possibly with different scalar `evaluation_factors`, to several target currents, the symbolic stage computes it once. For the worked graph this reduces 242 target attachments to 210 kernel evaluations. These three notions must remain distinct:

$$\underbrace{\text{same current key,}}_{\text{one stored node}} \quad \underbrace{\text{same evaluation group,}}_{\text{one symbolic kernel value}} \quad \underbrace{\text{same physical flow}}_{\text{one public output axis member}}. \quad (18)$$

10.2 Topological stages and value slots

The compiler sweeps proper source subsets in increasing cardinality. An interaction whose result covers k source labels belongs to recursion stage k . No evaluator can read a current from its own or a later stage, so the schedule is acyclic by construction. For this example,

$$\boxed{11 \text{ sources}} \longrightarrow \boxed{|I| = 2 : 18} \longrightarrow \boxed{|I| = 3 : 40} \longrightarrow \boxed{|I| = 4 : 48} \longrightarrow \boxed{48 \text{ amplitude roots}}. \quad (19)$$

For one concrete source-state path, the same schedule can be read as

$$E_2 : \quad C_{21} = \mathcal{P}_{21} K_{10}(C_2, C_9), \quad C_{25} = \mathcal{P}_{25} K_{14}(C_7, C_9), \quad (20)$$

$$E_3 : \quad C_{53} = \mathcal{P}_{53} [K_{66}(C_{21}, C_7) + K_{82}(C_2, C_{25}) + \dots], \quad (21)$$

$$E_4 : \quad C_{69} = \mathcal{P}_{69} [K_{90}(C_{53}, C_4) + \dots], \quad (22)$$

$$E_{\text{root}} : \quad \mathcal{A}_0 = \mathcal{C}_0(C_{69}, C_1). \quad (23)$$

Here C_2 is the label-2 quark source with helicity -1 , C_7 and C_9 are the negative-helicity gluon sources on labels 4 and 5, C_4 is the negative-helicity Z source, and C_1 is the endpoint antiquark source. The omitted terms are other admissible partitions contributing coherently to the same target. The subscripts on K are evaluation-group IDs, not current or interaction IDs.

```
sources: C2=d(2,-), C7=g(4,-), C9=g(5,-), C4=Z(3,-), C1=dbar(1,+)
E2: I10: C2 + C9 -> C21 [G10] I14: C7 + C9 -> C25 [G14]
E3: I66: C21 + C7 -> C53 [G66] I83: C2 + C25 -> C53 [G82]
      I82: C2 + C25 -> C61 [G82] (one kernel value, two targets)
E4: I98: C53 + C4 -> C69 [G90]
root: R0: contract(C69,C1) -> A0
```

This adjacency-list excerpt is diagnostic but exact for the current graph. It also exposes why the namespaces cannot be conflated: C_{53} is a canonical current node, I_{83} is one attachment to it, G_{82} identifies a reusable symbolic evaluation, and R_0 is an amplitude output. Value-slot IDs are a further runtime namespace assigning propagated or unpropagated component storage to currents. Internal LC sector IDs annotate root construction; public IDs such as `flow:2,5,4,1` label the resolved physics axis.

The stage records are assembled by `build_runtime_schema()` in `src/pyamplicol/generation/runtime_schema.py`. They list input/output current IDs, value-slot IDs, momentum slots, interaction IDs, and both attachment and unique-evaluation counts.

A current and its stored value are not identical notions. Depending on its consumers, one current can have a `source`, `unpropagated`, or `propagated` value slot. Each slot owns a contiguous component range; Weyl currents have two complex components and vector currents have four in the worked model. Keeping variants explicit prevents the runtime from silently applying a propagator twice and lets the amplitude closure request precisely the form it needs.

The complete schema for this graph has 256 complex value components, 13 four-vector momentum slots (52 real components), and 16 real model-parameter slots. A conceptual flattened state row therefore has

$$x = [x_{\text{value}} \mid x_{\text{momentum}} \mid x_{\text{model}}], \quad |x| = 256 + 52 + 16 = 324. \quad (24)$$

No compiled stage accepts all 324 entries. Stage-local layout is mandatory: for stage k , a map L_k selects only the current components, momentum components, and model parameters actually used there,

$$x_k = L_k x, \quad y_k = E_k(x_k), \quad x[S_k] \leftarrow y_k. \quad (25)$$

The construction is in `src/pyamplicol/generation/stage_parameters.py`; Rusticol stores the same local-to-global indices in the execution manifest. Contiguous index runs are precomputed, so most packing and output assignment becomes slice copies rather than scalar lookup loops.

10.3 What Symbolica and SymJIT compile

`build_generic_stage_compiler_blueprint()` in `src/pyamplicol/generation/stage_planning.py` creates one multi-output symbolic blueprint per recursion stage and one for the amplitude roots. For a target current j , its output expression is

$$y_j = \mathcal{P}_j \left[\sum_{e: \text{target}(e)=j} f_e K_{g(e)}(x_k) \right], \quad (26)$$

where $K_{g(e)}$ is cached by evaluation-group ID, f_e is the attachment factor, and $\mathcal{P}_j$ is the output propagator or identity. Component expressions, couplings, momentum dependence, and propagators come from the selected model; the stage compiler does not know that this example contains a Z or a gluon.

Symbolica optimizes the whole multi-output expression of one stage: common subexpressions, Horner schemes, and common-pair elimination can therefore cross current-component boundaries inside that stage. With the default JIT backend, SymJIT O3 then compiles a batched complex-f64 application. The direct application capability is recorded as `symjit.application.complex-f64.v1`; Rusticol can load and execute this f64 payload without importing Symbolica. Retained Symbolica evaluator state is used for higher-precision Python requests.

Output chunking is a compilation boundary, not a physics boundary. If a stage has more than the configured 512 scalar outputs, it is split into multi-output chunks sharing the same local input vector. A fanout-aware ordering in `src/pyamplicol/generation/stage_planning.py` places outputs with common evaluation groups near one another, reducing duplicated work when a group would otherwise cross chunk boundaries. At runtime the local input is packed once per stage and batch, each compiled chunk consumes that same packed buffer, and the concatenated outputs are scattered to their recorded slots. Dependencies are still cut only between recursion stages.

Compiled generation streams these stages. Once a blueprint has been compiled and written, its Symbolica expressions can be released before the next stage is built. This bounds peak expression memory without changing the DAG or the runtime schedule. The relevant writer path is `src/pyamplicol/generation/stage_artifacts.py`, while backend settings and payload construction live in `src/pyamplicol/evaluators/symbolica_compile.py`.

10.4 Artifact records and Rusticol execution

For the compiled route described in this section, the root `artifact.json` is a process-artifact schema-v3 manifest. It binds global model/configuration data and payload hashes to process-local records. The files relevant to one concrete process are:

record	runtime meaning
<code>processes/evaluators.json</code>	evaluator capabilities and payload locations
<code>processes/<id>/execution.json</code>	value/momentum/model layout, sources, ordered stages, chunks, roots, and reduction schedule
<code>processes/<id>/physics.json</code>	external particles, physical helicity and flow IDs, coverage, symmetry expansion, selectors, and normalization
<code>evaluator applications/libraries</code>	target-specific compiled numerical kernels
<code>API/</code>	validation point and Python, Rust, C++, and Fortran examples

The complete `CurrentIndex` is a generation contract. The execution projection serializes only the fields needed to validate and run its slots; ordered-label proofs and other generation-only evidence need not burden the hot runtime. By contrast, physical helicity and flow metadata remains explicit because it defines the public resolved axes and selector semantics.

Rusticol's f64 path is implemented in `rust/crates/rusticol-core/src/engine/evaluation.rs`. For a batch of points, one execution is:

1. reuse and clear the state scratch buffer;
2. fill source wavefunctions in their source value slots;
3. form all recorded all-outgoing momentum sums and copy current model parameter values into the global state;
4. for every recursion stage, pack x_k , call all evaluator chunks, and assign y_k to the global value slots;
5. pack and call the amplitude-root evaluator;
6. sum roots coherently, contract colour, expand exact physical symmetry representatives when resolved output is requested, and apply normalization.

The stage and amplitude implementations reuse their parameter and output scratch buffers across calls. The global model-parameter vector is mutable at runtime; an atomic update validates all supplied values, refreshes derived parameters and masses, and recomputes the factor in Eq. (16). No mass, width, or coupling is baked into the orchestration schedule merely because it is constant in one run card.

For LC resolved evaluation, the public tensor has shape

$$R_{n,h,c} = |\mathcal{M}_{h,c}(p_n)|^2, \quad \text{shape}(R) = (N_{\text{point}}, N_{\text{helicity}}, N_{\text{flow}}). \quad (27)$$

For this process the final two dimensions have lengths 48 and 2. Structural zeros occupy their declared entries. `evaluate_resolved()` can select stable helicity/flow IDs and materializes these components; its `total()` is their explicit sum. `evaluate()` follows the optimized summed path directly and need not allocate the resolved tensor. Neither interface exposes internal current, slot, or sector IDs.

10.5 The two runtime timing boundaries

The profiler deliberately distinguishes orchestration from compiled evaluator calls. Its reported f64 wall sample starts in the native Rusticol method after the caller-language buffer conversion and repeatedly executes the ordinary core path through final reduction. It therefore includes Rust-side input batch preparation, source and momentum construction, stage-local packing, chunk calls, output assignment, root packing/call, and reduction, but excludes Python/NumPy, C++, Fortran, or Rust caller-side packing.

The evaluator sum is narrower:

$$t_{\text{eval}} = \sum_k t(E_k) + t(E_{\text{root}}), \quad t_{\text{core}} = t_{\text{eval}} + t_{\text{source}} + t_{\text{momentum/model}} + t_{\text{pack}} + t_{\text{assign}} + t_{\text{reduce}} + t_{\text{batch}}. \quad (28)$$

provenance marker	quantity
<code>runtime_core_repeated_wall_time</code>	native repeated total path, normalized per point; common metric for all language APIs
<code>runtime_profile_core_evaluator_call_time</code> native component profile	sum of recursion-stage and amplitude-root evaluator call times only source fill, momentum/model setup, per-stage input pack, evaluator call and output assignment, root pack/call, and reduction

These boundaries make a wall/evaluator difference interpretable as work inside Rusticol rather than Python probing overhead. They also make Python, Rust, C++, and Fortran measurements comparable when they call the same f64 artifact. The CLI calibration, uncertainty estimate, progress display, and interruptible sampling policy are implemented in `src/pyamplicol/benchmarking.py`; the native timers themselves are in `rust/crates/rusticol-core/src/engine/native_runtime.rs` and `rust/crates/rusticol-core/src/engine/evaluation.rs`.

11 Reusable Leading-Colour Execution Layouts

Leading-colour coverage and leading-colour execution layout are separate contracts. Let $\mathcal{H}$ be the retained physical helicity IDs and let $\mathcal{F}$ be the retained physical colour-flow IDs. A complete LC artifact represents

$$\mathcal{C}_{\text{LC}} = \mathcal{H} \times \mathcal{F} \quad (29)$$

and can therefore expose every resolved component $\mathcal{A}_{hf}$, independently of how its recurrence is scheduled. For runtime-selected subsets $S_H \subseteq \mathcal{H}$ and $S_F \subseteq \mathcal{F}$, Rusticol computes

$$\mathcal{W}(S_H, S_F) = \mathcal{N} \sum_{h \in S_H} \sum_{f \in S_F} w_{hf} \left| \sum_{r \in G(h,f)} \mathcal{A}_{hf}^{(r)} \right|^2. \quad (30)$$

Here $G(h, f)$ is one coherent root group, w_{hf} contains the exact symmetry and LC weights, and $\mathcal{N}$ is the process normalization. A layout changes the representation and order used to evaluate these terms; it does not change $\mathcal{H}$, $\mathcal{F}$, or Eq. (30).

11.1 Two complete-coverage layouts

pyAmpliCol provides two LC layouts because the two principal workloads expose different opportunities for current sharing.

property	topology-replay	all-flow-union
Primary workload	one runtime-selected flow, summed helicities	all flows, one runtime-selected helicity
Retained physics	every physical flow and helicity	every physical flow and helicity
Main reuse	exact flow-topology representatives, label permutations, signs, and multiplicities	currents, interactions, and closures shared directly across the union of physical flows
Compiled lane	representative fused stage recurrences replayed by Rusticol	fused cross-flow union stages selected by runtime helicity
Eager lane	prepared-kernel invocations scheduled by replay class	prepared-kernel invocations lowered from the same cross-flow union
Default	yes	explicit opt-in

In the topology-replay layout, exact certificates partition physical flows into recurrence classes. If $\rho(f)$ is the representative of the class containing f , the artifact records a source-label permutation π_f , an exact sign or factor s_f , and its reduction weight. Schematically,

$$\mathcal{A}_{hf}(p) = s_f \mathcal{A}_{\pi_f(h), \rho(f)}(\pi_f(p)). \quad (31)$$

Rusticol maps the requested flow to its representative, groups compatible point rows, evaluates contiguous backend batches, and scatters the weighted result to the public flow ID. This avoids materializing one independent DAG for every physical ordering and is the layout used for the single-flow/helicity-sum benchmark.

The all-flow-union layout instead constructs the complete cross-flow current dependency union before numerical lowering. If D_{hf} is the dependency graph of one resolved component, its graph is the exact quotient

$$D_{\text{union}} = \left(\bigcup_{h \in \mathcal{H}, f \in \mathcal{F}} D_{hf} \right) / \sim, \quad (32)$$

where $\sim$ identifies currents, kernel evaluations, attachments, and closures whose canonical contracts prove them equal up to stored factors. For a selected helicity, one sweep of this union shares common intermediate currents across all requested flows. This is the layout used for the all-flows/single-helicity benchmark. The artifact does not also carry a second topology-replay graph or a generation-fixed-helicity graph.

Both layouts are available with compiled, eager, and recurrence execution. Compiled mode lowers the chosen graph to process-specific stage evaluators. Eager mode lowers it to invocation tables over prepared model kernels. Recurrence mode directly constructs compact current and contribution schedules for the selected layout without first materializing the generic DAG. Thus `execution_mode` selects how numerical kernels are supplied, while `lc_flow_layout` selects how complete LC recurrence work is shared.

11.2 Runtime selectors

Public selectors always use the stable IDs stored in the physics manifest, such as `flow:2,5,4,1` and `h:-1,+1,+0,-1,+1`. Batch-global `color_flows` and `helicities` select rectangular subsets of Eq. (29). The per-point `color_flow_by_point` and `helicity_by_point` inputs select one component on the corresponding axis for each point. A global and a per-point selector are mutually exclusive on the same axis. Omitting an axis sums every retained component on that axis.

Rusticol stably groups per-point selector rows before numerical evaluation so that equal selectors form contiguous SIMD-compatible packets. An already grouped input remains in order and does not pay for a redundant sort. This selector planning is a runtime operation; it does not reduce the generated coverage of either layout. Deliberately generation-sliced artifacts remain a separate expert facility and, by construction, can select only components that were retained during their generation. The all-flow-union layout requires complete flow and helicity coverage and therefore rejects such generation slices.

The layout is selected in a run card with

```
[color]
accuracy = "lc"
lc_flow_layout = "all-flow-union"
```

or with `-lc-flow-layout all-flow-union`. Omitting the option keeps the `topology-replay` default.

11.3 Why the report has two generation times

The two layouts retain the same physical component set but build different graphs, evaluator payloads, and runtime schedules. Their process-generation times are consequently distinct observables,

$$T_{\text{gen}}^{\text{replay}} \quad \text{and} \quad T_{\text{gen}}^{\text{union}}, \quad (33)$$

and neither value is copied into the other workload's table slot. The LC selected-flow/helicity-sum columns use $T_{\text{gen}}^{\text{replay}}$; the all-flows/single-helicity columns use $T_{\text{gen}}^{\text{union}}$. Original **AmpliCol** produces one table-driven library for both workloads, so its one generation time remains the denominator of both recurrence/**AmpliCol** ratios. In the built-in cross-mode tables, each compiled or eager generation is compared with recurrence using the same LC layout. Prepared-model creation is a reusable model-level operation and remains outside eager and recurrence process-generation timers.

11.4 NLC and full colour

The layout option applies only to LC. NLC and full-colour artifacts already construct one complete amplitude basis and contract it with a sparse colour metric,

$$\mathcal{W}_{\text{contracted}} = \mathcal{N} \sum_h \sum_{a,b} \mathcal{A}_{ha}^* C_{ab} \mathcal{A}_{hb}. \quad (34)$$

They do not expose the physical LC-flow axis whose alternative scheduling motivates topology replay and the all-flow union. Configuration therefore rejects `all-flow-union` for NLC and full colour. Their generation, contraction, cache, and runtime contracts are unchanged by this choice.

12 Prepared-Model Eager DAG Execution

pyAmpliCol provides two execution contracts. The default *compiled* mode turns a complete process stage into one or more process-specific evaluator applications. The opt-in *eager* mode compiles the model-local vertex, current-finalization, closure, and parameter kernels once, then represents a process by compact invocation tables. Rusticol interprets those tables in its numeric core. The modes use the same physical DAG, normalization, colour reduction, selectors, and public APIs, but deliberately place the compilation boundary at different levels:

	Compiled mode	Eager mode
Compiled unit	A process-stage expression containing many current recurrences	One canonical model-local kernel evaluation class
Process generation	Builds and compiles process-specific stage applications	Rust lowers proven DAG columns to invocation, finalization, closure, reduction, and selector tables
Runtime scheduling	Calls a small number of wide process-stage applications	Gathers packets, calls reusable kernels, scatters contributions, and finalizes currents
Coverage contract	Complete artifacts support runtime selectors; selected-coverage artifacts may specialize	The same complete-coverage and selected-coverage contracts
Default	Yes	No; selected with <code>execution_mode = "eager"</code>

Eager execution is not a replacement for compiled execution. It is a second lane for applications where rapid process generation, reusable model kernels, or runtime selector flexibility matters more than maximal process-wide kernel fusion. No eager branch is present in the compiled lane’s inner loop, and the compiled artifact contract remains unchanged.

12.1 Prepared model and process plan

A prepared model bundle separates model work from process work. Schematically, the bundle is

$$\mathcal{B}_{M,b,o,t} = \left(\mathcal{I}_M, \mathcal{E}_M, \{K_{\kappa}^{(b,o,t)}\}_{\kappa \in \mathcal{K}_M}, \mathcal{P}_M \right), \quad (35)$$

where $\mathcal{I}_M$ is the portable compiled-model IR, $\mathcal{E}_M$ retains exact expressions, K_{κ} is one prepared kernel for canonical evaluation class κ , and $\mathcal{P}_M$ contains ABI and provenance metadata. The labels b, o, t identify backend, optimization settings, and target contract. A bundle contains exactly one prepared JIT, C++, or ASM backend pack.

Eager process generation keeps symbolic and numerical responsibilities separate. Python performs the Symbolica algebra, physics proofs, prepared-kernel resolution, and extraction of the proven **GenericDAG**. It then fills fixed-width columnar arrays for currents, interactions, roots, kernel references, permutations, aliases, selector proofs, and reductions. Rust validates those columns and lowers them directly to

$$\mathcal{Q}_P = (\{\text{Inv}_s, \text{Att}_s, \text{Fin}_s\}_{s=1}^S, \text{Close}, \text{Reduce}, \text{Select}). \quad (36)$$

Here Inv_s identifies kernel calls in recursion stage s , Att_s maps their outputs to current components, Fin_s applies the current’s finalization or propagator once, Close builds amplitudes, and Reduce performs the physical colour/helicity contraction. **Select** stores dependency domains for runtime selectors. The Rust lowering assigns slots, interns selector domains, groups invocations, schedules attachments and finalizations, constructs closures and coherent reductions, validates the resulting plan, and writes the indexed binary runtime layout. It does not return expanded Python row objects or construct a large JSON runtime schema.

No backend evaluator is built or compiled during this process step. Invocation rows refer to kernels already present in $\mathcal{B}$, and the referenced prepared applications are copied into the standalone process artifact. The cost of preparing $\mathcal{B}$ is therefore a separate model-level quantity, not part of eager process-generation time.

Prepared kernels are derived from the same verified oriented-kernel catalog used by the model-generic UFO lowering. The built-in Standard Model emits that same catalog, while its explicitly Standard-Model-specific physics remains under `src/pyamplicol/models/builtin/`. Couplings, colour coefficients, exchange signs, input permutations, and proof factors are invocation metadata, not distinct compiled applications. Existing proof-gated auxiliary-current decompositions continue to reduce supported higher-point interactions to the same local kernel classes described here and in the UFO-support section below.

12.2 Stage execution

Let $V_{c\alpha p}$ be component α of current slot c at phase-space point p . An invocation row r stores a kernel identifier $\kappa(r)$, input slots and component permutations, momentum slots, coupling references, an exact factor w_r , and an output attachment. For one stage, Rusticol first forms unpropagated current components

$$U_{c\alpha p}^{(s)} = \sum_{r \in \mathcal{R}_s(c, \alpha)} w_r [K_{\kappa(r)}(G_r(V_{\cdot p}), q_r(p), \theta)]_{\alpha(r)}. \quad (37)$$

The gather map G_r is fixed by the invocation table. Contributions are accumulated before current finalization, so a current with momentum Q_c is materialized exactly once:

$$V_{c\beta p}^{(s)} = [F_{\phi(c)}(U_{c\cdot p}^{(s)}, Q_c(p), \theta)]_{\beta}. \quad (38)$$

The finalization class $\phi(c)$ may be an identity, a propagator, or another model-proved current transformation. This ordering prevents a propagator from being applied once per contributing vertex and preserves the compiled-mode normalization convention.

After the last recursion stage, closure rows produce amplitude components,

$$A_{h\gamma p} = \sum_{r \in \mathcal{C}(h, \gamma)} w_r^{\text{cl}} K_{\kappa(r)}^{\text{cl}}(G_r^{\text{cl}}(V_{\cdot p}), q_r(p), \theta), \quad (39)$$

and the existing LC or contracted-colour reduction computes the requested total or resolved observable. In a contracted basis this has the schematic form

$$\mathcal{M}_p = \mathcal{N} \sum_h \sum_{\gamma, \delta} A_{h\gamma p} C_{\gamma\delta} A_{h\delta p}^*, \quad (40)$$

with the same colour matrix C , symmetry factors, initial-state average, and coupling normalization as compiled mode.

The Rust implementation is deliberately separate from compiled execution. Plan loading and validation are in `rust/crates/rusticol-core/src/eager_runtime/plan.rs`; the allocation-free numeric loop is in `eager_runtime/execute.rs`; runtime ownership and workspace reuse are in `eager_runtime/runtime.rs`. The artifact-facing integration is isolated under `rusticol-core/src/engine/eager_*.rs`.

12.3 Packetization, tiling, and allocation discipline

Rusticol does not call a backend once per DAG edge. Invocation rows are packetized by kernel class and compatible mapping. For a tile of T points, the innermost layout keeps points contiguous,

$$X[\text{argument}][\text{invocation}][\text{point}], \quad (41)$$

so a SymJIT application can auto-vectorize across points. C++ and ASM packs receive the same batched packet but remain scalar across rows; no SIMD claim is made for those backends.

Current storage is component-major with points contiguous. Gather buffers, kernel outputs, current storage, selector masks, closure storage, and reduction workspace are allocated at load or selector change and then reused. After warm-up, `evaluate_into` performs no heap allocation in the Rust numeric core. Large batches are tiled using

$$T_{\text{eff}} = \min\left(T_{\text{requested}}, \left\lfloor \frac{W_{\text{limit}}}{W_{\text{per point}}} \right\rfloor\right), \quad (42)$$

subject to a minimum valid packet. The public defaults are a requested tile size of 1024 points and a 256 MiB workspace. This keeps memory bounded for arbitrarily large Monte-Carlo batches without placing Python or NumPy packing inside the reported Rusticol wall time.

Eager profiling separates initialization, gather, kernel calls, invocation scatter, finalization, finalization scatter, closure, reduction, and copy-out. Those markers are returned by `pyamplicol profile` and the same native runtime profile used by Python, C11, C++17, Fortran 2008, and Rust 2021 clients. The report's eager `eval` field is the inclusive eager execution lane, not kernel-call time alone; the detailed profile separates its constituent phases.

12.4 Runtime selectors and complete LC coverage

For a selector σ , eager lowering stores a dependency domain $D(x)$ for each invocation, attachment, finalization, and closure row x . The active schedule is

$$\mathcal{Q}_P(\sigma) = \{x \in \mathcal{Q}_P : D(x) \cap \sigma \neq \emptyset\}, \quad (43)$$

including all transitive current dependencies. Masks and packet schedules are rebuilt only when a selector changes, not at every phase-space point.

This distinction is important for LC timing. Every *complete* eager LC artifact contains each physical flow and generated helicity component. A caller may select any one flow for a helicity sum, one helicity for all flows, or any supported subset at runtime. The flow identifier, for example `flow:3,1,2,5,6,7,4`, is therefore a runtime choice rather than a flow hard-wired during process generation. A deliberately selected-coverage eager artifact remains limited to the components that were generated.

Compiled mode exposes the same complete-coverage runtime contract. Coverage does not, however, prescribe one execution graph. As described in Sec. 11, both execution modes provide topology-replay and all-flow-union layouts. The report uses topology replay for single-flow/helicity-sum and the all-flow union for all-flows/single-helicity, with separate process-generation times. Either artifact can choose any retained flow or helicity without regeneration. Deliberately specialized artifacts remain useful as optimization baselines, but are not the reusable report workloads.

12.5 Artifact, precision, and portability contracts

An eager process artifact is standalone. It copies each referenced prepared kernel exactly once under the artifact root and never depends on the original prepared-model path. Fixed-width little-endian tables use 32-bit identifiers and 64-bit offsets/counts. Plan-v3 artifacts use runtime kind `pyamplicol-runtime-eager-execution`, capability `rusticol.eager-runtime-layout.complex-f64.v1`, and four complementary records:

- a small `execution.json` summary containing identity, ABI, provenance, counts, digests, options, and inspection statistics;
- a compact `physics.json` containing public process axes, coverage, selector metadata, parameters, normalization, and proof summaries;
- `processes/<id>/eager-runtime.pacbin`, containing the process-local invocation, slot, selector, closure, reduction, and proof tables; and
- the artifact-root `evaluators.pacbin`, containing indexed prepared evaluators and their exact states.

High-cardinality runtime data is not duplicated in JSON. Rusticol authenticates the artifact and PACBIN indices, memory-maps the containers, and decodes the eager layout into a validated owned execution plan. Normal `inspect` operations use the summary metadata rather than scanning every table. The loader rejects malformed offsets, unknown kernel identifiers, incompatible ABIs, and target mismatches before entering native evaluation.

This representation reduces Python object construction, JSON serialization, filesystem entry count, artifact footprint, peak generation memory, and cold load work. It does not change the eager numerical schedule: topology replay, LC all-flow union, NLC/full contraction, aliases, structural zeros, and global or per-point selectors retain their existing semantics. Both LC layouts use the same Rust lowering and compact container contract. Exact Python execution reads the same indexed plan and evaluator states. Target-native C++ and ASM libraries remain separate payloads.

Plan-v3 is the eager artifact compatibility boundary. Earlier eager artifacts use an expanded runtime representation and must be regenerated; the loader identifies that capability before opening the old high-cardinality payload and reports the required regeneration. This requirement is limited to eager artifacts and does not change compiled artifacts or prepared-model bundles.

JIT packs store SymJIT application state rather than final machine code. The portable SymJIT storage v3 contract is available at optimization level 2, and the receiving host rebuilds executable code for its CPU. Wheels therefore carry the built-in-SM `built-in-sm-jit-o2` prepared pack. pyAmpliCol forces O2 when preparing JIT kernels, including when another level was requested. This does not change process-local compiled DAG evaluators, whose independent configuration may use O3. Prepared C++ and ASM packs remain target-native.

Symbolica is required during eager generation and for non-f64 exact execution. After generation, saved JIT applications support Symbolica-free f64 execution, including multicore use. Exact expressions remain in the artifact, and `src/pyamplicol/runtime/eager_exact/` executes the same plan for double-double and arbitrary precision. Thus the execution-mode choice does not change the public precision contract.

12.6 Performance interpretation and scaling

Eager process-generation timing begins with an already prepared model bundle and ends with an authenticated, loadable process artifact. It includes Python proof/DAG extraction, transfer-column construction, Rust plan lowering, compact container writing, and post-build validation. Prepared-model creation is reported separately. Useful comparisons therefore report process generation, prepared-model creation, artifact size, peak resident memory, authenticated cold load, and first-use break-even as distinct quantities.

Rust table lowering and high-cardinality serialization handle the part of eager generation that grows with the process DAG. Compact PACBIN storage also avoids materializing equivalent expanded Python and JSON objects. The resulting benefit depends on current count, selector-domain complexity, colour contraction, and proof metadata; it is not represented by one universal speedup. Published process and *Z*-ladder tables provide the measured values under the benchmark contract defined in the main methodology section.

For runtime, eager mode avoids process-specific backend compilation but pays explicit gather, scatter, finalization, and reusable-kernel dispatch. SymJIT amortizes those costs over point batches through internal SIMD. Selector measurements distinguish homogeneous, already grouped, alternating, and random per-point selections because regrouping and SIMD occupancy can change the wall time without changing the matrix element. A separately specialized compiled stage can remain faster for some selectors; fusing entire recurrence regions would instead reproduce the compiled mode's process-specific applications.

12.7 Choosing the mode

Use compiled mode when process generation is affordable and the target workload benefits from process-wide specialization. Use eager mode when many processes share one model, when generation time dominates, or when one artifact must support changing LC flow/helicity selections. The common built-in-SM JIT O2 prepared-kernel path needs only

```
pyamplicol generate "d d~ > z g g g" artifacts/ddbar_z3g_eager \
--model built-in-sm --execution-mode eager --color-accuracy nlc
```

External models and C++/ASM backends first require an explicit `pyamplicol model compile` prepared bundle. Missing or mismatched packs fail before DAG construction with the exact preparation command; eager generation never silently compiles a backend pack.

13 Arbitrary UFO and Serialized-JSON Models

The external-model path is a compiler, not a permissive UFO interpreter. It accepts a trusted UFO Python module or its serialized JSON representation, normalizes the model to a typed, model-independent intermediate representation, and only then exposes interactions to the current-recursion generator. A structure is executable when the compiler can construct its component expressions and prove every transformation used to lower or reuse them. Failure to prove a useful optimization merely disables that optimization; failure to prove a transformation required for correctness rejects the model with a structured diagnostic.

This section describes the implemented contract. The handwritten Standard Model in `src/pyamplicol/models/builtin/` is deliberately outside that contract: `BuiltinSMModel` may use SM-specific formulae, while `CompiledUFOModel` must derive its decisions from particle metadata, normalized tensors, propagators, couplings, and exact algebra. The boundary forbids SM PDG classifiers and built-in implementation symbols from leaking into the external-model modules.

13.1 One route from UFO and JSON to an executable model

The public entry point `compile_model_source()` in `src/pyamplicol/models/loading.py` recognizes a UFO directory, a JSON model, an already compiled model, or the special built-in model. For an external source it lazily invokes `ufo-model-loader`. Both source forms therefore enter the same route:

$$\begin{aligned} \{\text{UFO directory, serialized JSON}\} &\xrightarrow{\text{load_model}} \text{ufo-model-loader.Model} \\ &\xrightarrow{\text{compact serialization and compilation}} \text{CompiledModelIR}. \end{aligned} \quad (44)$$

The UFO directory is executable Python and must therefore be trusted. The JSON form is data-only and is the portable form to archive or exchange.

The loader is first called with wrapped Lorentz indices disabled. This lets `_classify_external_propagators()` compare a model-supplied propagator against the loader’s exact default Feynman-gauge candidates. Indices are then wrapped and the model is serialized with `JSONLook.COMPACT`. Consequently, UFO and JSON inputs do not maintain separate physics implementations after loading.

Every source symbol is put in a model-local namespace by `src/pyamplicol/_internal/physics/symbols.py`. A model called `sm` uses heads such as `model_sm::GC_1`, while compiler-owned currents, momenta, dummy indices, and derived couplings use the `pyamplicol::` namespace. The registry rewrites the loader’s temporary `UFO::` heads; a compiled `CompiledModelIR` is rejected if an executable expression still contains a process-global `UFO::` symbol. Two loaded models can thus coexist without silently identifying equally named parameters or tensors.

The canonical IR in `src/pyamplicol/models/contracts.py` contains typed records for particles, parameters, coupling orders, propagators, vertex terms, oriented trivalent kernels, direct and closure contractions, tensor and current orderings, and Goldstone partners. It is not a bag of strings. On reload it checks the model-schema version, compiler version and fingerprint, proof identities, tensor-ordering links, and required fields before any DAG is built. The compiler fingerprint includes the `pyAmpliCol`, `Symbolica`, and `ufo-model-loader` versions and the relevant serialization and lowering policies; a stale compiled model is regenerated rather than guessed compatible.

compiler stage	resulting contract	implementation
source loading	one compact loader-model payload	<code>models/loading.py: _load_external_model()</code> and <code>preflight_model()</code>
parameter resolution	namespaced external and derived expressions	<code>models/compiler_records.py</code> and <code>ModelSymbolRegistry</code>
tensor normalization	canonical colour and Lorentz expressions	<code>models/tensors.py</code> , using <code>idenso</code> , <code>Spenso</code> , and <code>Symbolica</code>
kernel construction	oriented trivalent contracts and contact auxiliaries	<code>models/compiler_entry.py</code> , <code>models/compiler_kernels.py</code> , <code>models/compiler_contacts.py</code>
ordering construction	explicit source axes, Weyl projections, and current embeddings	<code>models/compiler_tensor_ordering.py</code>
model-owned closure	propagators, direct/closure contractions, and Goldstone policy	<code>models/compiler_contractions.py</code> , <code>models/compiler_gauge.py</code>
optimization proofs	kernel equivalence, colour-flow auxiliaries, and exact model symmetries	<code>models/compiler_color_flow.py</code> and <code>models/external_symmetries.py</code>

13.2 From a UFO vertex to concrete current kernels

A UFO vertex is a matrix over colour and Lorentz structures. The compiler expands it into separate terms

$$\mathcal{V}(1, \dots, n) = \sum_{a,b} g_{ab} C_a(c_1, \dots, c_n) L_b(s_1, p_1; \dots; s_n, p_n), \quad (45)$$

and stores the original sources together with their normalized forms and the coupling-order vector of g_{ab} . The normalization in `src/pyamplicol/models/tensors.py` converts wrapped UFO heads to typed Spenso expressions. It applies idenso's `simplify_metrics`, `simplify_gamma`, and `simplify_color`; Spenso then materializes dense tensor components in a declared Cartesian order. Residual UFO tensor heads, indeterminate expressions, duplicate coordinates, or missing dense coordinates are errors.

The normalized Lorentz basis includes scalars, momenta, metrics, Dirac identity and gamma matrices, γ_5 , chiral projectors, slashed momenta, and sigma tensors. The supported colour representations are the SU(3) singlet, fundamental, antifundamental, and adjoint. Exact trilinear projection recognizes four executable colour families:

$$1, \quad \delta_i^{\bar{j}}, \quad (T^a)_i^{\bar{j}}, \quad f^{abc}. \quad (46)$$

For a proposed basis tensor B_σ , projection succeeds only when the normalized tensor obeys the componentwise identity

$$C_a(\mathbf{c}) = \kappa_a B_\sigma(\mathbf{c}) \quad \text{for every materialized colour component } \mathbf{c}, \quad (47)$$

with an exact scalar κ_a . Numerical projection exists as a diagnostic but does not certify an executable kernel. The symmetric adjoint invariant d^{abc} , for example, can be recognized diagnostically but is not in the current executable trilinear colour basis.

For a trilinear term, `_compile_oriented_kernels()` chooses each distinct result species and both nonidentical input orders. Spenso contracts the normalized tensor with formal left and right currents and physical momenta. The resulting component contract has the schematic form

$$K_{r \leftarrow ij}^{A_r}(J_i, J_j) = g_{ab} \kappa_a \sum_{A_i, A_j} L_b^{A_r A_i A_j}(p_i, p_j, p_r) J_i^{A_i} J_j^{A_j}, \quad p_r = -(p_i + p_j), \quad (48)$$

followed by the model-owned propagator when the result is an internal current. The coupling, perturbative orders, runtime parameters, source-leg map, colour normalization, and input/output ordering identifiers remain attached to the kernel. Weyl projection is therefore an explicit ordering/embedding contract, not an inference made later from a particle name.

The compiler recognizes a compact Yang–Mills three-vector form only after its components have been proved equal, up to one exact constant, to the dense Spenso result. This is an implementation optimization, not an alternative physics formula.

13.3 What a proof or certificate asserts

The word *proof* is used narrowly. It is a reproducible algebraic witness that an optimized or decomposed executable contract reconstructs the normalized source contract. It is not a statement that a model is physically consistent, unitary, or gauge complete. Three assurance mechanisms are distinct:

1. A **serialized decomposition proof** is attached to a four-point `CompiledVertexTerm`. Its algorithm name and version, source identity, split topology, index map, component basis, signs, and multiplicities are revalidated when the compiled model is loaded.
2. **Kernel-equivalence metadata** is written on oriented kernels. Fully lowered component expressions are compared under input exchange and an exact factor $\eta = \pm 1$. Equal kernels receive one `evaluation_class`, a representative input order, and a factor, so the DAG may evaluate the representative once and fan its value out.
3. A **model-level symmetry certificate** is re-derived by `derive_external_symmetry_certificates()` when a `CompiledUFOModel` is constructed. It binds a theorem to digests of the compiled particle, propagator, ordering, vertex, and executable-kernel contracts. It is not accepted as a free-form claim from the UFO source.

For two kernel expressions K and K' , reusable evaluation requires an exact relation

$$K'(x, y; p_x, p_y) = \eta K(y, x; p_y, p_x), \quad \eta \in \{-1, +1\}, \quad (49)$$

after resolving coupling aliases and canonicalizing all component expressions. For a global theorem, the digest is

$$D = \text{SHA256}(\text{canonicalJSON}\{\text{source, propagator, orderings, terms, kernels}\}). \quad (50)$$

The selectors and 64-character digests must be complete and mutually consistent. Renaming a contracted tensor dummy must not change D , but a different contraction graph must. The internal helper

`_canonical_tensor_product_expressions()`

in `src/pyamplicol/models/external_symmetries.py` calls Symbolica's `Expression.canonize_tensors()` on the contracted Lorentz, colour, and spinor indices. Its reserved replacement head is

`pyamplicol::canonical_tensor_index.`

The certificate contract requires alpha-renaming invariance and contraction-graph sensitivity.

The important fail-closed distinction is:

- if a reflection or parity certificate is absent, the generator keeps both configurations and remains correct, but performs less reuse;
- if a four-point contact cannot be lowered without changing its tensor, colour, coupling, or multiplicity, generation stops rather than silently dropping or approximating that interaction.

proof class	checked identity	optimization or lowering enabled
trilinear colour projection	Eq. (47) in the supported SU(3) basis	colour-flow interpretation of an oriented kernel
kernel evaluation equivalence	Eq. (49), including coupling and ordering contracts	one evaluator value reused by symmetry-related current contributions
four-point contact decomposition	exact colour topology, open-axis Lorentz basis, orientation signs, and multiplicity	two trivalent auxiliary-current stages with no artificial pole
fundamental Fierz identity	exact T^a trilinear projection and eligible massless adjoint-vector metadata	compiler-owned singlet-subtraction auxiliary for LC flow
vectorlike parity	exact fundamental-generator colour and gamma-current Lorentz tensor	global helicity-flip representative reuse
Yang–Mills cubic/quartic sector	exact f^{abc} , three-vector tensor, quartic basis, coupling relation, and closed reachable sector	pure-adjoint helicity-zero theorem and LC trace-reflection folding
local adjoint reflection	every relevant cubic kernel is antisymmetric under left/right exchange	current reflection phase reuse
Goldstone partner contract	quantum numbers, exact mass expression, uniqueness, and propagator policy	unitary-gauge absorption or explicit model-supplied Goldstone treatment
tensor/current ordering	explicit physical axes and component permutations	stable Spenso-to-current and Weyl component maps

13.4 Four-point contacts: exact auxiliary-current factorization

Every momentum-independent valence-four term receives a `CompiledContactDecompositionProof` before lowering. The serialized algorithm is `ufo-four-point-contact-decomposition`, version 2. A proof has status `proven` or `unsupported`; its identity includes the term and vertex identifiers, particle tuple, colour and Lorentz matrix indices, original sources, and normalized expressions. A stale version or an identity mismatch is rejected.

The record family is explicit:

- `CompiledContactDecompositionProof` owns the term identity and all proved result-leg splits;
- `CompiledContactDecompositionSplit` and `CompiledContactDummyIndexMapping` describe one topology and its shared colour index;
- `CompiledContactOrientationProof` fixes every input order and sign; and
- `CompiledContactUnsupportedReason` carries a stable failure code and context.

Two decomposition kinds are implemented.

Literal colour singlet. If all four legs are colour singlets and $C = 1$, the Lorentz tensor may be split directly. A literal singlet attached to any coloured leg is not treated as an implicit contraction; it fails with `literal-singlet-with-colored-legs`.

Product of two structure constants. The coloured gauge-contact path requires exactly

$$C^{a_1 a_2 a_3 a_4} = c f^{a_i a_j b} f^{b a_k a_r}, \quad (51)$$

up to exact permutations, with one and only one shared adjoint dummy b and a scalar prefactor c . For each possible result leg r , the compiler records the paired legs (i, j) , remaining leg k , dummy-index map, permutation parity, LC normalization powers, and the exact scalar coefficient. It then introduces a non-propagating auxiliary current

$$A_{\rho\sigma}^b[i, j] = f^{b a_i a_j} \sum_{A_i, A_j} P_{\rho\sigma; A_i A_j} J_i^{A_i} J_j^{A_j}, \quad (52)$$

$$J_r^{A_r}|_{V_4} = \frac{c s_{ijk r}}{m_r} f^{a_r b a_k} \sum_{\rho, \sigma, A_k} F^{A_r}_{\rho\sigma A_k} A_{\rho\sigma}^b J_k^{A_k}. \quad (53)$$

Here $s_{ijk r} = \pm 1$ is the recorded orientation parity and m_r is the number of indistinguishable assignments of the remaining input species. The original coupling and complete perturbative-order vector occur only in Eq. (53); Eq. (52) has unit coupling and no propagator. The factorization therefore introduces neither an unphysical pole nor a duplicated coupling/order weight.

The Lorentz split is not inferred from the phrase “four-gluon vertex.” Spenso first contracts the chosen input pair while leaving the result and remaining-leg axes open. `_compress_contact_components()` stores a basis of distinct components and reconstructs every omitted component by an exact zero or ± 1 map. The final kernel contracts that basis and must reproduce the source tensor component by component. This same mechanism covers an SU(3) four-vector term such as the four-gluon vertex and colour-singlet electroweak-like four-vector contacts built from metric tensors. What differs is the colour proof: `two-structure-constants` for the former and `literal-color-singlet` for the latter.

The compiler deduplicates identical contact partials and fuses compatible finals after proof validation. Unsupported diagnostics distinguish an incorrect topology from a missing feature: wrong factor count, malformed structure-constant indices, non-scalar colour prefactor, non-unique shared dummy, absent result leg, and a nontrivalent structure-constant topology all have separate reason codes in `src/pyamplicol/models/compiler_contacts.py`.

13.5 Arbitrary-valence colour-singlet contact trees

Colour-singlet contacts with valence $n \geq 4$ are lowered by `_compile_color_singlet_contact_trees()` in `src/pyamplicol/models/compiler_contact_trees.py`. The already proved momentum-independent four-point case uses the preceding path; other singlet contacts are split recursively at the midpoint into a balanced binary tree. For a set of input leaves S , the generic recursion is

$$A_S = \Phi_S(A_{S_L}, A_{S_R}), \quad S = S_L \dot{\cup} S_R, \quad J_r|_{V_n} = \frac{g}{m_r} \Psi_r(A_{S_L}, A_{S_R}), \quad (54)$$

where the internal A_S are non-propagating, g and its coupling orders appear only in the final map Ψ_r , and m_r removes identical-input assignment multiplicity. A balanced tree has $O(n)$ auxiliary operations and $O(\log n)$ depth.

For an all-scalar contact, each auxiliary carries one component and $\Phi_S(x, y) = xy$. This is the path exercised by the arbitrary-valence scalar-contact model. For non-scalar or momentum-dependent singlet contacts, an auxiliary carries the current components and four-momentum of every physical leaf below it. At the final node, the original normalized n -point Lorentz expression is reconstructed with

$$p_r = - \sum_{i \neq r} p_i. \quad (55)$$

On Linux, very large dense contractions are evaluated in bounded stages when the direct input work exceeds 1024 components; each staged group is limited to 256 input components and its intermediate tensor is materialized before the next contraction. This changes only contraction order. The direct function `eager_color_singlet_vertex_term_components()` remains the component oracle for the tree result.

This is deliberately not advertised as an arbitrary coloured n -point algorithm. It is generic in valence, spin, momentum dependence, and Lorentz expression only after the colour tensor has been proved to be a singlet that requires no unresolved coloured internal channel.

13.6 Colour-flow and model-level symmetry proofs

The LC compiler derives its singlet-subtraction channel from the fundamental Fierz identity

$$(T^a)_i^{\bar{j}} (T^a)_k^{\bar{l}} = T_R \left(\delta_i^{\bar{l}} \delta_k^{\bar{j}} - \frac{1}{N_c} \delta_i^{\bar{j}} \delta_k^{\bar{l}} \right). \quad (56)$$

The compiler function

```
synthesize_fundamental_fierz_auxiliaries()
```

in `src/pyamplicol/models/compiler_color_flow.py` is enabled only for an exactly projected T^a trilinear with fundamental fermion lines and a self-conjugate, massless adjoint vector with a standard propagator. It creates one compiler-owned colour-singlet vector auxiliary and reuses the proved Lorentz and coupling kernel. The $1/N_c$ closure coefficient is attached in one canonical orientation. No gluon name or PDG code participates.

Four further exact recognizers in `src/pyamplicol/models/external_symmetries.py` recover the reusable structure normally associated with a gauge theory:

- A **vectorlike parity kernel** has one massless adjoint vector, a fundamental/antifundamental fermion pair, exact T^a colour, and a Lorentz tensor that is a constant multiple of the gamma-current reference. Chiral-projector deformations do not pass this proof.
- A **Yang–Mills cubic kernel** has one self-conjugate massless adjoint-vector species, a standard propagator, exact f^{abc} , and the three-vector Lorentz tensor up to an exact constant. The effective cubic coupling is retained symbolically.
- A **Yang–Mills quartic group** must uniquely span the three $ff \times gg$ basis elements of the four-vector vertex, and each coefficient must obey the exact relation to the square of the cubic coupling. Recognizing three individually familiar-looking terms is insufficient if the coupling relation fails.
- A **local adjoint-current reflection** requires every relevant cubic kernel to satisfy Eq. (49) with $\eta = -1$, exact f^{abc} , and a standard propagator.

A global Yang–Mills theorem is emitted only if the reachable trivalent compiled sector is closed: starting from the candidate adjoint source, every kernel reachable from two already reachable currents must itself be certified Yang–Mills. Contact auxiliaries are included in this traversal. One extra reachable non-Yang–Mills interaction therefore disables the global theorem rather than being ignored. A complete certificate enables the pure-adjoint helicity-zero theorem, global helicity-flip representatives, LC trace reflection, and shared single-trace basis. The ordinary DAG remains complete when the certificate is absent.

These generic proofs recover the built-in-SM current topology for the established UFO-SM validation matrix without assuming that every external model must do so. Validation covers electroweak, QCD, top, pure-gluon, charged-current, multi-boson, and up to three-quark-line examples. Equivalent built-in and UFO-SM models must have

equal minimal coupling limits and equal current, interaction, amplitude-root, and colour-sector counts; exact kernel reuse may reduce evaluator calls but may not increase them. The same proof is invariant under arbitrary particle-name, PDG, and inventory-order changes. This establishes the certified classes, not a universal topology theorem for all UFO syntax.

13.7 Propagators, closures, Goldstones, and spin two

Propagators are model contracts rather than hard-coded particle identities. For standard scalar, Dirac, vector, and spin-two records, the evaluator in `src/pyamplicol/models/external_evaluation.py` implements the expected runtime-mass and runtime-width kernels, schematically

$$P_0(p) = \frac{i}{p^2 - m^2 + im\Gamma}, \quad P_{1/2}(p) = \frac{i(\gamma^\mu p_\mu + m)}{p^2 - m^2 + im\Gamma}, \quad (57)$$

$$P_1^{\mu\nu}(p) = \frac{-i}{p^2 - m^2 + im\Gamma} \left(g^{\mu\nu} - \frac{p^\mu p^\nu}{m^2} \right), \quad P_2^{\mu\nu, \rho\sigma}(p) = \frac{i \Pi^{\mu\nu, \rho\sigma}(p)}{p^2 - m^2 + im\Gamma}. \quad (58)$$

The massless-vector branch is Feynman gauge and does not use the singular $p^\mu p^\nu / m^2$ term. Weyl currents use explicit chirality projections. For spin two, Π is the de Donder massless or Fierz–Pauli massive projector represented in a 16-component ordered tensor space. Synthetic contact auxiliaries have the identity map and no propagator; the Fierz singlet-subtraction auxiliary propagates as its compiler-owned massless vector.

A custom UFO propagator is accepted when its normalized numerator and denominator can be materialized through the same Spenso ordering contract. It is applied exactly and is not silently converted to a preferred gauge. Its presence intentionally disables symmetry theorems that require the standard propagator.

`compile_contraction_records()` emits explicit scalar, Dirac, opposite Weyl-chirality, mostly-minus vector, and rank-two tensor-product-metric contractions. Rusticol consumes these records and does not infer a closure from component count. Goldstone matching in `compile_goldstone_partner_records()` similarly uses quantum-number flow and the exact resolved mass expression. A unique massive vector with a default propagator absorbs its Goldstone in the unitary-gauge path; a custom vector propagator retains the model-supplied Goldstone. Ambiguous matches fail closed.

UFO spin code 5 is therefore a real current type, not a special-case scalar: it has two ordered Lorentz axes, 16 components, tensor-product closure, and a spin-two propagator. The packaged massless scalar-gravity model validates the generic tensor route, including axis relabelling and transposed source storage. Massive spin two is accepted with an explicit experimental warning; arbitrary massive spin-two phenomenology is not yet a qualified release capability.

13.8 Implemented support boundary

The support matrix below separates implemented current capabilities from unsupported classes. “Proof-gated” means that qualifying models work without an SM-specific exception; it does not mean every algebraically equivalent UFO spelling is already recognized.

area	class	status	exact boundary
source	UFO directory / JSON	supported	both lower through one loader-model payload; UFO Python must be trusted
spin	scalar, Dirac, vector	supported	standard wavefunctions, propagators, current orderings, and closures
spin	massless spin two	supported	16-component tensor path tested by the packaged scalar-gravity model
spin	massive spin two	experimental	Fierz–Pauli path exists, but broad model validation is incomplete
spin	UFO ghosts	partial	one-component metadata and ordering exist; propagating ghosts require an exactly lowerable custom propagator and are not broadly qualified
spin	Majorana/FNV, spin 3/2	unsupported	no fermion-number-violating flow or Rarita–Schwinger current contract
colour	1, 3, $\bar{3}$, 8 of SU(3)	supported	LC and current contracted NLC/full basis

area	class	status	exact boundary
colour arity 3	$6, \bar{6}$, extra groups $1, \delta, T, f$ colour	unsupported supported	no sextet or multi-group flow/contraction representation exact projection plus generic materializable Lorentz components
arity 3	d^{abc} , ϵ , K_6 , T_6	unsupported	recognized or known source heads are not sufficient to construct an executable colour contract
arity 4	colour singlet	supported	momentum-independent proof path; generic singlet-tree path for other materializable tensors
arity 4	two- f gauge colour	proof-gated	one shared adjoint dummy, trivalent topology, scalar prefactor, exact Lorentz reconstruction
arity 4 arity > 4	other coloured tensors colour-singlet contact	unsupported supported	no certified auxiliary colour channel balanced no-pole tree; original coupling/order only at final node
arity > 4 Lorentz	coloured contact idenso/Spenso basis	unsupported supported	no general irreducible-channel factorization proof all source tensor heads must normalize and materialize with explicit axes
Lorentz	form factors / unknown functions	unsupported	no executable and serializable function contract
propagator propagator	standard $0, \frac{1}{2}, 1, 2$ custom UFO tensor	supported proof-gated	runtime mass/width and explicit ordering/closure exact Spenso lowering; gauge conversion and affected symmetry reuse are disabled
symmetry	vectorlike/Yang–Mills/reflection	proof-gated	exact normalized identities and complete reachable-sector digest

Preflight in `preflight_model()` rejects unsupported spin or colour codes, Majorana fermions, form factors, wrong known-function arities, and unknown expression functions before expensive compilation. Tensor normalization and contact lowering impose the narrower executable boundary after preflight. Thus recognizing a UFO head such as `K6` during syntax inspection does not imply that a sextet colour current can be built.

13.9 Support boundary and extension requirements

Support is defined by algebraic contracts rather than process-family or PDG exceptions. A new class requires a typed source representation and normalization, an explicit component ordering and current embedding, a direct contraction and colour-flow interpretation, and an exact proof of orientation signs, coupling placement, multiplicities, and any evaluator reuse. The proof must remain invariant under dummy-index, source-label, and particle-name renaming, and its resolved sum must agree with a direct or higher-precision oracle. Without those contracts the model is rejected at the corresponding boundary rather than assigned an inferred implementation.

unsupported or restricted class	required contract
additional scalar UFO functions adjoint d^{abc} trilinears epsilon and sextet tensors	define a serializable Symbolica lowering and derivatives/branch semantics where relevant add a physical colour basis, projection, contraction, and LC/NLC/full interpretation add sextet current identities, conjugation, flow basis, and sparse colour metric before tensor lowering
other coloured four-point contacts	prove a complete set of auxiliary irreducible channels and exact reconstruction, including repeated representations and multiplicities
coloured valence above four	general representation-channel tree with associativity/basis-change proofs and controlled intermediate growth
Majorana/FNV fermions	fermion-flow orientation, charge conjugation, signs, wavefunctions, propagators, and closure contracts
spin 3/2	ordered Rarita–Schwinger wavefunctions, projectors, contractions, and tensor-component qualification
multiple colour groups	group-qualified representations, independent invariant tensors, product flow bases, and contracted colour metrics
general massive spin two	broader model and gauge/constraint validation around the existing 16-component propagator contract

Validation includes successful examples and deliberately deformed tensors, couplings, propagators, index orderings, and proof records. This negative coverage is part of the advertised support boundary: an unfamiliar model either satisfies the same explicit algebraic contract or receives a diagnostic where a new proof is required.